شيت 4: الحلقات التكرارية

شرح الأسئلة من 1 لـ 16 بلغة C++

الصفحة دي بتشرح أسئلة شيت 4 بتاع الأسبوع الخامس في GEN241، سؤال سؤال. كل سؤال بيبدأ بنصه زي ما هو في الشيت، وبعدين معناه بالظبط، والخلفية اللي محتاجها من رياضة أو غيرها، وبعدين بنبني الكود حتة صغيرة ورا حتة: كود قصير، والخرج اللي طلع منه فعلاً، وليه طلع كده، وبعدين نغيّر حاجة واحدة بس ونشوف إيه اللي اتغير. في الآخر البرنامج كامل، وطريقة تانية للحل، والأخطاء اللي ناس كتير بتقع فيها.

اقروا كل سؤال من فوق لتحت بالترتيب، وجرّبوا الأكواد الصغيرة بنفسكم قبل ما تبصوا على الخرج، بالطريقة اللي في الجزء الجاي. ولو عايزين تروحوا لسؤال معين، استخدموا القايمة اللي تحت.

إزاي تشغّلوا كود صغير

الأكواد الصغيرة اللي في الأسئلة مش برامج كاملة؛ هي السطور اللي بتتكتب جوه `main` بس. علشان تشغلوا أي واحد منها، اعملوا ملف جديد وحطوا فيه الإطار ده:

```
#include<iostream>
#include<iomanip>
#include<cmath>
#include<cctype>
using namespace std;

int main()
{
    // one short example here
    return 0;
}
```

وبعدين حطوا مثال واحد بس مكان السطر `// one short example here`، وشغّلوا. المثال اللي بعده يتحط في برنامج جديد من نفس الإطار، مش تحت اللي قبله. ده مهم لسببين: أمثلة كتير بتعرّف متغيرات بنفس الأسماء (زي `x` و`count`)، والتنسيق اللي بنحطه على `cout` زي `fixed` و`setprecision` بيفضل شغال لحد آخر البرنامج، فلو جمعتوا مثالين في برنامج واحد، الأول ممكن يغير شكل خرج التاني. ولو تحت الكود فيه «الإدخال»، البرنامج هيستناكم تكتبوه وتدوسوا Enter. الإطار فيه كل المكتبات اللي الأمثلة الصغيرة محتاجاها، وكل مثال صغير في الصفحة اتجرب جوه الإطار ده بالظبط وطلّع نفس الخرج المكتوب.

إزاي اتعمل الخرج اللي هنا

كل خرج في الصفحة دي متسجل من تشغيل حقيقي، مش مكتوب باليد. الأمثلة الشغالة والبرامج الكاملة مكتوبة بأسلوب المحاضرة ومعيار C++98، واتترجمت بإعدادات صارمة بتعتبر أي تحذير خطأ، وما طلعش ولا تحذير.

وفيه أمثلة غلط مقصودة علشان نشوف المشكلة بعنينا، ودي فوقها علامة حمرا فيها كلمة «غلط» وسبب الغلط. معظمها بيترجم عادي وبيطلّع خرج غلط، وحاجة واحدة بس (في السؤال 7) مذكورة في الكلام من غير تشغيل لأن المترجم بيرفضها خالص. وفيه أمثلة فوقها

علامة رمادي (للمقارنة، أو تجربة، أو خطوة في النص): شغالة، لكنها مش الحل المطلوب، زي طريقة حل المصدر أو أرقام الشيت أو مرحلة لسه ناقصة.

التشغيل كان على Mac بمترجم clang؛ ما اتجربش على Windows ولا على Dev-C++ نفسه. الحسابات هي هي، لكن شكل حاجات قليلة جداً، زي كلمة `inf` في السؤال 6، ممكن يختلف من مترجم للتاني.

في البرامج اللي بتقرا من الكيبورد، إحنا بعتنا الإدخال للبرنامج جاهز من ملف. علشان كده اللي اتكتب مش ظاهر جنب رسالة السؤال في الخرج المسجل، وبنكتبه لوحده تحت كلمة «الإدخال». لما تشغلوا البرنامج بنفسكم وتكتبوا من الكيبورد في نافذة تشغيل عادية، اللي بتكتبوه بيظهر عادةً بعد الرسالة، وبعده سطر جديد لما تدوسوا Enter. ولو في الإدخال مسافة مهمة، هتلاقوها مكتوبة بالعلامة `␣`.

المصادر الأصلية: الشيت (sheet-04.pdf) والمحاضرة (lecture-06.pdf).

الأسئلة

1. عدّ الموجب والسالب، والمجموع والمتوسط لحد الصفر
2. جدول تحويل من ملليمتر لبوصة
3. جدول تحويل من متر لقدم
4. جدولين جنب بعض: ملليمتر وبوصة
5. جدولين جنب بعض: متر وقدم
6. جدول tan و cot من 0 لـ 60 درجة
7. جدول الدالة exp من 0 لـ 10
8. من 1 لـ 500: يقبل القسمة على 3 و 4
9. من 300 لـ 400: على 3 ومش على 6
10. من كود ASCII للحرف
11. من الحرف لكود ASCII
12. من حرف كبير لحرف صغير
13. تصنيف حرف واحد
14. الحروف من A لـ Z، خمسة في كل سطر
15. أول خمس مضاعفات لعدد
16. هرم الأرقام

---

1 عدّ الموجب والسالب، والمجموع والمتوسط لحد الصفر

(1) Write a C++ program that reads an unspecified number of integers, determines how many positive and negative values have been read, and computes the total and average of the input values (not counting zeros). Your program ends with the input 0. Display the average as a floating-point number.

المطلوب

البرنامج بيقرا أعداد صحيحة ورا بعض، ومحدش قاله من الأول هيقرا كام عدد. اللي بيوقف القراية هو إن المستخدم يكتب `0`. الصفر هنا مش رقم من البيانات، هو إشارة معناها «خلصت». الإشارة دي اسمها *sentinel*.

بعد ما القراية تقف، البرنامج يطبع أربع حاجات: عدد الأعداد الموجبة، وعدد الأعداد السالبة، ومجموع الأعداد، ومتوسطها. والشيت بيقول صراحة إن الأصفار مش داخلة في الحساب، وإن المتوسط يتعرض كرقم عشري (floating-point)، يعني ممكن يبقى فيه كسر.

مثال هنمشي عليه: لو المستخدم كتب `3` وبعدين `-1` وبعدين `4` وبعدين `0`، يبقى عندنا عددين موجبين وعدد سالب واحد، والصفر هو آخر حاجة وما بيتعدّش.

الخلفية

عددها ÷ مجموع القيم = المتوسط. في المثال، القيم هي 3 و 1− و 4. مجموعهم 6، وعددهم 3، يبقى المتوسط 2. خدوا بالكم إن «العدد» و«المجموع» حاجتين مختلفتين: العدد بيزيد واحد مع كل قيمة مهما كانت، أما المجموع فبيزيد بقيمة الرقم نفسه، وممكن يقل لو الرقم سالب.

وفيه حالة لازم نفكر فيها: لو أول حاجة اتكتبت هي الصفر، يبقى ما فيش ولا قيمة. ساعتها عدد القيم صفر، والقسمة على صفر ملهاش معنى في الرياضة. يعني المتوسط في الحالة دي مش موجود أصلاً، والبرنامج لازم يقول كده بدل ما يقسم.

من C++ هنحتاج

حلقة `do-while` علشان نقرا لحد الصفر، وعدادات بتزيد بـ `++`، ومتغير بيتجمع فيه المجموع، و `if / else if / else` علشان نفرق بين الصفر والموجب والسالب، وتحويل لـ `float` قبل القسمة علشان الكسر ما يضيعش. كل حاجة من دول هتظهر في مثال صغير تحت.

نبني الحل خطوة خطوة

> كل كود صغير في الجزء ده بيتجرب لوحده: حطوه مكان السطر `// one short example here` جوه `main` في الإطار اللي في أول الصفحة، في برنامج جديد لكل مثال. ولو تحت الكود فيه «الإدخال»، اكتبوه لما البرنامج يستناكم.

قراية رقم واحد

نبدأ بأصغر حاجة: نقرا رقم واحد، ونزوّد العداد واحد، ونضيف الرقم للمجموع.

```
int number, count = 0, sum = 0;
cin >> number;
count++;
sum = sum + number;
cout << "count = " << count << ", sum = " << sum << endl;
```

الإدخال

```
3
```

الخرج

```
count = 1, sum = 3
```

العداد بدأ من `0` وبقى `1` علشان قرينا رقم واحد. المجموع بدأ من `0` وبقى `3` علشان الرقم اللي اتقرا قيمته 3. لازم نكتب `= 0` للاتنين من الأول. المتغير المحلي اللي اتعرّف من غير قيمة اسمه uninitialized، وقيمته مش معرّفة، فممنوع نقراه قبل ما نحط فيه قيمة. و `count++` بتقرا القيمة القديمة قبل ما تزودها، فلو مفيش قيمة أولى يبقى البرنامج غلط.

نكرر لحد الصفر

دلوقتي نحط نفس الخطوة جوه `do-while`، ونطبع حالة العداد والمجموع بعد كل قراية علشان نتابع بعنينا. الإدخال هو المثال بتاعنا.

```
int number, count = 0, sum = 0;
do
{
    cin >> number;
    if (number != 0)
    {
        count++;
        sum = sum + number;
    }
    cout << "read " << number << ": count = " << count
         << ", sum = " << sum << endl;
}
while (number != 0);
```

الإدخال

```
3 -1 4 0
```

الخرج

```
read 3: count = 1, sum = 3
read -1: count = 2, sum = 2
read 4: count = 3, sum = 6
read 0: count = 3, sum = 6
```

بصوا على السطر التاني: العداد بقى `2`، لكن المجموع نزل لـ `2`، لأن 2 = (−1) + 3. ده بالظبط الفرق بين العدّ والجمع. في السطر الأخير اتقرا الصفر، والعداد والمجموع ما اتغيروش، لأن الزيادة كلها جوه `if (number != 0)`.

ليه `do-while` بالذات؟ لأن الشرط `number != 0` بيتسأل عن رقم لازم يكون اتقرا الأول. في `do-while` جسم الحلقة بيتنفذ مرة قبل أي اختبار، فبنقرا، وبعدين في آخر اللفة نسأل: الرقم اللي لسه قاربينه صفر؟ لو أيوه نخرج، لو لأ نلف تاني.

نغيّر حاجة واحدة: نشيل الـ if

تعالوا نشوف الصفر بيعمل إيه لو سبناه يتعد. نفس الكود ونفس الإدخال، بس شلنا الشرط، وزودنا سطر بيحسب المتوسط في الآخر.

غلط مقصود: العداد بيعدّ الصفر

```
int number, count = 0, sum = 0;
do
{
    cin >> number;
    count++;
    sum = sum + number;
    cout << "read " << number << ": count = " << count
         << ", sum = " << sum << endl;
}
while (number != 0);
cout << "average = " << (float)sum / count << endl;
```

الإدخال

```
3 -1 4 0
```

الخرج

```
read 3: count = 1, sum = 3
read -1: count = 2, sum = 2
read 4: count = 3, sum = 6
read 0: count = 4, sum = 6
average = 1.5
```

المجموع لسه `6`، لأن إضافة صفر ما بتغيرش حاجة. لكن العداد بقى `4` بدل `3`، والمتوسط طلع `1.5` بدل `2`. يعني الصفر ممكن يدخل المجموع من غير ضرر، لكن ممنوع يدخل العدد. البرنامج الكامل تحت بيستفيد من النقطة دي: بيجمع الرقم قبل ما يسأل هو صفر ولا لأ، لكنه ما بيعدّش الصفر.

نفصل الموجب عن السالب

السؤال عايز عدد الموجب لوحده وعدد السالب لوحده. فبدل عداد واحد نعمل اتنين، ونسأل بالترتيب: صفر؟ موجب؟ وإلا يبقى سالب.

```cpp
int number, positiveCount = 0, negativeCount = 0;
do
{
    cin >> number;
    if (number == 0)
        cout << "zero: stop" << endl;
    else if (number > 0)
        positiveCount++;
    else
        negativeCount++;
}
while (number != 0);
cout << "positive = " << positiveCount
     << ", negative = " << negativeCount << endl;
```

الإدخال

```
3 -1 4 0
```

الخرج

```
zero: stop
positive = 2, negative = 1
```

الـ `else` الأخيرة مش محتاجة شرط، لأن الرقم الصحيح اللي مش صفر ومش أكبر من صفر لازم يكون سالب. وعدد القيم الكلي، اللي هنقسم عليه، هو مجموع العدادين: `nonZeroCount = positiveCount + negativeCount` .

القسمة: فين بيضيع الكسر

المثال `3 -1 4 0` متوسطه 2 بالظبط، فمش هيبيّن مشكلة القسمة. نزود رقم: `3 -1 4 1 0` . المجموع كده 7 والعدد 4. نجرب القسمة لوحدها:

غلط للمتوسط: قسمة أعداد صحيحة

```cpp
int sum = 7, count = 4;
cout << sum / count << endl;
```

الخرج

```
1
```

الاتنين `int` ، فـ C++ بتعمل قسمة أعداد صحيحة: 7 على 4 بتدّي `1` ، والكسر بيتشال خالص، مش بيتقرب.

نغير حاجة واحدة: نحوّل `sum` لـ `float` قبل القسمة.

```
int sum = 7, count = 4;
cout << (float)sum / count << endl;
```

الخرج

```
1.75
```

دلوقتي واحد من الطرفين `float` ، فالقسمة بقت قسمة عشرية والنتيجة `1.75` . ولو حطينا التحويل بعد القسمة:

غلط للمتوسط: التحويل جه بعد القسمة

```
int sum = 7, count = 4;
cout << (float)(sum / count) << endl;
```

الخرج

```
1
```

الأقواس خلت `sum / count` تتحسب الأول كقسمة صحيحة وتطلع 1، وبعدين حولنا الـ 1 لـ `float` . التحويل متأخر، والكسر كان ضاع خلاص. علشان كده البرنامج بيكتب `(float)totalSum/nonZeroCount` من غير أقواس حوالين القسمة.

لما ما يبقاش فيه ولا قيمة

لو أول حاجة اتكتبت صفر، العدادين هيفضلوا صفر. البرنامج بيسأل قبل القسمة: `if (nonZeroCount == 0)` ، ولو أيوه بيطبع إن المتوسط مش معرّف وما بيقسمش خالص. ده تشغيل البرنامج الكامل بإدخال صفر بس:

الإدخال

```
0
```

الخرج

```
Enter an integer, the input ends if it is 0: You entered 0. The input is halted.

Number of positive integers entered: 0
Number of negative integers entered: 0
Total sum of all integers: 0
No nonzero values; average is undefined.
```

البرنامج كامل

دلوقتي نجمع الحتت مع بعض. فيه سطر واحد ما شرحناهوش لسه: `if (!(cin >> number)) return 0;` . معناه: حاول تقرا رقم؛ لو القراية نجحت كمّل عادي، ولو فشلت (اتكتب حرف بدل رقم مثلاً، أو الإدخال خلص) اقفل البرنامج. لما القراية تفشل، يبقى مفيش رقم جديد اتقرا نقدر نعتمد عليه، و `cin` بيفضل في حالة فشل، فالقرايات اللي بعدها كمان مش هتنجح لوحدها. من غير السطر ده، البرنامج كان هيكمّل كأن فيه رقم اتقرا: ممكن يحسب على قيمة مالهاش معنى، أو يفضل يلف ويحاول يقرا ويفشل.

```cpp
#include<iostream>
using namespace std;

int main()
{
    int number, positiveCount=0, negativeCount=0, nonZeroCount, totalSum=0;
    float average;

    // 1. read and classify each value until the sentinel
    do
    {
        cout << "Enter an integer, the input ends if it is 0: ";
        if (!(cin >> number)) return 0;
        totalSum = totalSum + number;
        if(number == 0)
        {
            cout << "You entered 0. The input is halted." << endl;
        }
        else if(number > 0)
        {
            positiveCount++;
        }
        else
            negativeCount++;
    }
    while(number!=0);
    nonZeroCount = positiveCount+negativeCount;

    // 2. report the counts and total before checking the denominator
    cout << "\nNumber of positive integers entered: " << positiveCount << endl;
    cout << "Number of negative integers entered: " << negativeCount << endl;
    cout << "Total sum of all integers: " << totalSum << endl;
    // 3. divide only when at least one nonzero value was entered
    if (nonZeroCount == 0)
        cout << "No nonzero values; average is undefined." << endl;
    else
    {
        average = (float)totalSum/nonZeroCount;
        cout << "Average: " << average << endl;
    }
    return 0;
}
```

التشغيل بالمثال بتاعنا. الرسالة بتتكرر أربع مرات لأن فيه أربع قرايات، والصفر آخرهم:

الإدخال

```
3 -1 4 0
```

الخرج

```
Enter an integer, the input ends if it is 0: Enter an integer, the input ends if it is 0: Enter
an integer, the input ends if it is 0: Enter an integer, the input ends if it is 0: You entered
0. The input is halted.

Number of positive integers entered: 2
Number of negative integers entered: 1
Total sum of all integers: 6
Average: 2
```

وبالإدخال اللي فيه كسر، المتوسط طلع `1.75` زي ما شفنا في التجربة الصغيرة:

الإدخال

```
3 -1 4 1 0
```

الخرج

```
Enter an integer, the input ends if it is 0: Enter an integer, the input ends if it is 0: Enter
an integer, the input ends if it is 0: Enter an integer, the input ends if it is 0: Enter an
integer, the input ends if it is 0: You entered 0. The input is halted.

Number of positive integers entered: 3
Number of negative integers entered: 1
Total sum of all integers: 7
Average: 1.75
```

لو ما طلبناش تنسيق، `cout` بيعرض الرقم العشري بستة أرقام معنوية (significant digits) بالكتير، ولو الرقم محتاج أقل بيعرضه بأقل، زي `1.75` فوق. هنا المجموع 7 والعدد 3، والكسر ما بيخلصش، فاتعرض بالستة أرقام كلهم:

الإدخال

```
3 -1 5 0
```

الخرج

```
Enter an integer, the input ends if it is 0: Enter an integer, the input ends if it is 0: Enter
an integer, the input ends if it is 0: Enter an integer, the input ends if it is 0: You entered
0. The input is halted.

Number of positive integers entered: 2
Number of negative integers entered: 1
Total sum of all integers: 7
Average: 2.33333
```

وأعداد سالبة بس: العداد الموجب صفر، والمتوسط سالب عادي.

الإدخال

```
-2 -4 0
```

الخرج

```
Enter an integer, the input ends if it is 0: Enter an integer, the input ends if it is 0: Enter
an integer, the input ends if it is 0: You entered 0. The input is halted.

Number of positive integers entered: 0
Number of negative integers entered: 2
Total sum of all integers: -6
Average: -3
```

حل تاني: `while (true)` مع `break`

بدل ما الاختبار يبقى في آخر اللفة، ممكن نعمل حلقة شرطها دايماً صحيح، ونخرج منها بإيدينا أول ما نقابل الصفر. ده الجزء اللي اتغير:

```
while (true)
{
    cout << "Enter an integer, the input ends if it is 0: ";
    if (!(cin >> number)) return 0;
    totalSum = totalSum + number;
    if(number == 0)
    {
        cout << "You entered 0. The input is halted." << endl;
        break;
    }
    else if(number > 0)
    {
        positiveCount++;
    }
    else
        negativeCount++;
}
```

`while (true)` لوحدها مش هتقف أبداً، والـ `break` هي اللي بتخرّجنا من الحلقة لحظة ما الصفر يتقرا، وبعدها التنفيذ بيكمل من أول سطر بعد الحلقة. الفرق في الفكرة: في `do-while` قرار الخروج بيتاخد في آخر اللفة بعد ما الـ `if` خلصت، وهنا بيتاخد جوه فرع الصفر نفسه. الخرج هو هو:

الإدخال

```
3 -1 4 1 0
```

الخرج

```
Enter an integer, the input ends if it is 0: Enter an integer, the input ends if it is 0: Enter
an integer, the input ends if it is 0: Enter an integer, the input ends if it is 0: Enter an
integer, the input ends if it is 0: You entered 0. The input is halted.

Number of positive integers entered: 3
Number of negative integers entered: 1
Total sum of all integers: 7
Average: 1.75
```

البرنامج البديل كامل

```cpp
#include<iostream>
using namespace std;

int main()
{
    int number, positiveCount=0, negativeCount=0, nonZeroCount, totalSum=0;
    float average;

    // 1. read and classify each value until the sentinel
    while (true)
    {
        cout << "Enter an integer, the input ends if it is 0: ";
        if (!(cin >> number)) return 0;
        totalSum = totalSum + number;
        if(number == 0)
        {
            cout << "You entered 0. The input is halted." << endl;
            break;
        }
        else if(number > 0)
        {
            positiveCount++;
        }
        else
            negativeCount++;
    }
    nonZeroCount = positiveCount+negativeCount;

    // 2. report the counts and total before checking the denominator
    cout << "\nNumber of positive integers entered: " << positiveCount << endl;
    cout << "Number of negative integers entered: " << negativeCount << endl;
    cout << "Total sum of all integers: " << totalSum << endl;
    // 3. divide only when at least one nonzero value was entered
    if (nonZeroCount == 0)
        cout << "No nonzero values; average is undefined." << endl;
    else
    {
        average = (float)totalSum/nonZeroCount;
        cout << "Average: " << average << endl;
    }
    return 0;
}
```

ممكن تقع في ده

- **تعدّ الصفر.** لو العداد بيزيد قبل ما تتأكد إن الرقم مش صفر، هتقسم على 4 بدل 3، وزي ما شفنا المتوسط يطلع `1.5` بدل `2`.
- **تقسم `int` على `int`.** `totalSum / nonZeroCount` لوحدها بتشيل الكسر حتى لو هتخزن النتيجة في `float`. التخزين بيحصل بعد ما القسمة خلصت.
- **تحط التحويل في المكان الغلط.** `(float)(totalSum / nonZeroCount)` بيحوّل نتيجة اتقطعت خلاص. حوّل واحد من الطرفين قبل القسمة.

- **تختبر بمثال متوسطه رقم صحيح.** `3 -1 4 0` بيطلع 2 في الحالتين، فمش هيكشف غلطة القسمة. جرّب دايماً إدخال مجموعه ما بيتقسمش على العدد بالظبط.
- **تقسم وما فيش بيانات.** لو الصفر أول حاجة، العدد صفر. اسأل قبل القسمة.
- **تبدأ بـ `while (number != 0)` قبل أي قراية.** ساعتها الشرط بيسأل عن متغير لسه ما خدش قيمة. إما تقرا قبل الحلقة، أو تستخدم `do-while`.

ملاحظة على حل المصدر

حل المصدر بيقسم على عدد القيم من غير ما يسأل هو صفر ولا لأ. لو أول إدخال صفر، النتيجة مش رقم (اتسجلت `nan` على المترجم اللي اتجرب عليه). البرنامج هنا زوّد سؤال قبل القسمة وسطر بيقفل البرنامج لو القراية فشلت. باقي المنطق زي حل المصدر بالظبط.

---

2 جدول تحويل من ملليمتر لبوصة

(2) Write a C++ program that displays the following table (note that $1\ inch = 25.4\ millimeters$):

```
Millimeters     Inches
2               0.078
4               0.156
...
96              3.744
98              3.822
```

المطلوب

جدول فيه عمودين. العمود الشمال فيه ملليمترات، والعمود اليمين فيه نفس الطول بالبوصة (inch). من الجدول اللي في الشيت بنعرف المدى: الملليمترات بتبدأ من 2، وبتزيد 2 كل سطر (2، 4، ...، 96، 98). يعني الأعداد الزوجية من 2 لحد 98، وده 49 سطر.
والمعلومة الوحيدة اللي في السؤال: البوصة الواحدة فيها 25.4 ملليمتر.

الجدول في الشيت بيعرض البوصات بتلات أرقام بعد العلامة العشرية، فهنعرضها كده.

الخلفية

البوصة أكبر من الملليمتر، والبوصة الواحدة فيها 25.4 ملليمتر. يعني لو معاك عدد ملليمترات وعايز تعرف دول كام بوصة، بتسأل: «كام مرة 25.4 داخلة في العدد ده؟». وده سؤال قسمة:

```
inches = millimeters / 25.4
```

طريقة تتأكدوا بيها إن الاتجاه صح: اكتبوا الوحدات. بوصة = (ملليمتر لكل بوصة) ÷ ملليمتر، لأن المللليمتر بيروح مع المللليمتر. وكمان تأكيد سريع بالعقل: عدد البوصات لازم يطلع أصغر من عدد المللليمترات، لأن البوصة وحدة أكبر.

من C++ هنحتاج

حلقة `for` عدّادها بيزيد 2، وقسمة بتطلّع رقم عشري، وتلات أدوات من `<iomanip>` بتتحكم في شكل الرقم وهو بيتطبع: `fixed` و`setprecision` و`setw`، ومعاهم `left`. الأدوات دي جديدة وفيها تفاصيل، فهنقف معاها شوية بأمثلة صغيرة قبل ما نرجع للجدول.

نبني الحل خطوة خطوة

> كل كود صغير في الجزء ده بيتجرب لوحده: حطوه مكان السطر `// one short example here` جوه `main` في الإطار اللي في أول الصفحة، في برنامج جديد لكل مثال. ولو تحت الكود فيه «الإدخال»، اكتبوه لما البرنامج يستناكم.

تحويل رقم واحد

```
int mm = 2;
cout << mm / 25.4 << endl;
```

الخرج

```
0.0787402
```

`mm` نوعه `int`، لكن `25.4` رقم عشري، فالقسمة بقت قسمة عشرية والنتيجة فيها كسر. و`cout` لما ما نقولوش حاجة بيعرض الرقم بستة أرقام معنوية بالكتير. الرقم ده كسره طويل، فاتعرض بالستة كلهم: `0.0787402`. والأصفار اللي في الأول بعد العلامة ما بتتحسبش من الستة. ولو الرقم محتاج أرقام أقل، بيتعرض بأقل، زي ما هنشوف في السؤال 3.

الرقم صح، بس الجدول عايز تلات أرقام بعد العلامة. هنا ييجي دور التنسيق.

وقفة مع التنسيق: `fixed` و`setprecision`

التنسيق بيظبط شكل الرقم وهو بيتطبع. نبدأ برقم سهل:

```
double x = 12.5;
cout << fixed << setprecision(2) << x << endl;
```

الخرج

```
12.50
```

`fixed` معناها اعرض الرقم بالشكل العشري العادي، و `setprecision(2)` مع `fixed` معناها اعرض رقمين بالظبط بعد العلامة العشرية. علشان كده 12.5 طلعت `12.50` .

نغير حاجة واحدة بس: الـ 2 تبقى 3.

```
double x = 12.5;
cout << fixed << setprecision(3) << x << endl;
```

الخرج

```
12.500
```

كمّل بصفر زيادة علشان يظهر تلات أرقام بعد العلامة زي ما طلبنا. نرجع لـ 2، ونغير الرقم نفسه بس لرقم فيه أرقام كتير بعد العلامة:

```
double x = 12.3456;
cout << fixed << setprecision(2) << x << endl;
```

الخرج

```
12.35
```

12.3456 فيها أربع أرقام بعد العلامة، وإحنا طالبين اتنين بس. فـ C++ قرّبت: اللي بعد الرقم التاني هو 56، ودول بيقربوا التاني لفوق، فطلعت `12.35` مش `12.34` . لو طلبنا تلاتة:

```
double x = 12.3456;
cout << fixed << setprecision(3) << x << endl;
```

الخرج

```
12.346
```

ولو طلبنا صفر أرقام بعد العلامة، التقريب بيحصل برضه، والعلامة نفسها بتختفي:

```
double x = 12.8;
cout << fixed << setprecision(0) << x << endl;
```

الخرج

```
13
```

طب لو نسينا `fixed` ؟ الرقم اللي جوه `setprecision` بيتغير معناه (القيمة المخزنة نفسها ما بتتغيرش):

من غير `fixed` : مش ده اللي عايزينه في الجدول

```
double x = 12.3456;
cout << setprecision(3) << x << endl;
```

الخرج

```
12.3
```

من غير `fixed` ، الـ 3 بقت عدد الأرقام المعنوية كلها، قبل العلامة وبعدها، مش عدد الأرقام بعد العلامة. علشان كده طلع `12.3` .

وقفة مع التنسيق: عرض الخانة بـ `setw`

المسافات صعب تشوفوها، فهنحط `|` قبل الرقم وبعده علشان نشوف حدود اللي اتطبع:

```
double x = 12.5;
cout << "|" << fixed << setprecision(2) << x << "|" << endl;
```

الخرج

```
|12.50|
```

نزود `setw(8)` قبل الرقم، والباقي زي ما هو:

```
double x = 12.5;
cout << "|" << fixed << setprecision(2) << setw(8) << x << "|" << endl;
```

الخرج

```
|   12.50|
```

ليه ظهرت 3 مسافات؟ لأن `12.50` واخدة 5 أماكن، وإحنا طالبين خانة عرضها 8، فكمّل الباقي مسافات على الشمال. الرقم نفسه ما اتغيرش، اللي اتغير هو الخانة اللي اتحط فيها.

نغير حاجة واحدة: نزود `left` .

```
double x = 12.5;
cout << "|" << left << fixed << setprecision(2) << setw(8) << x << "|" << endl;
```

الخرج

```
|12.50   |
```

نفس الخانة ونفس الرقم، بس الرقم راح ناحية الشمال والمسافات بقت بعده. ده اللي البرنامج بيستخدمه في الجدول علشان الأعمدة تبدأ من نفس المكان.

ولو الرقم أطول من الخانة؟

```
double x = 123456.75;
cout << "|" << fixed << setprecision(2) << setw(8) << x << "|" << endl;
```

الخرج

```
|123456.75|
```

الرقم واخد 9 أماكن والخانة 8، ومع كده اتطبع كامل. `setw` بتحدد أقل عرض، عمرها ما بتقص الرقم.

مين بيفضل شغال ومين بيخلص بعد مرة

هنا أهم نقطة في التنسيق. نطبع رقمين ورا بعض بعد `setw(8)` واحدة:

```
cout << "|" << fixed << setprecision(2) << setw(8) << 12.5 << 7 << "|" << endl;
```

الخرج

```
|   12.507|
```

الـ 7 لزقت في `12.50` من غير مسافات. `setw` بتأثر على الحاجة اللي بعدها مباشرة بس، وبعدها بترجع لأصلها. ولاحظوا كمان إن الـ 7 طلعت `7` مش `7.00`، لأنها `int`، و`setprecision` بتأثر على الأرقام العشرية بس. علشان الرقم التاني ياخد خانة، لازم نكتب `setw` قبله هو كمان:

```
cout << "|" << fixed << setprecision(2) << setw(8) << 12.5 << setw(8) << 7 << "|" << endl;
```

الخرج

```
|   12.50       7|
```

أما `fixed` و`setprecision` فبيفضلوا شغالين على `cout` لحد ما نغيرهم. نطبع نفس المتغير مرتين، ونكتب التنسيق في المرة الأولى بس:

```cpp
double x = 12.3456;
cout << fixed << setprecision(2) << x << endl;
cout << x << endl;
```

الخرج

```
12.35
12.35
```

المرة التانية طلعت `12.35` برضه من غير ما نكتب أي تنسيق. ده معناه حاجة مهمة: التنسيق بيغير شكل العرض بس، والمتغير لسه جواه 12.3456. علشان نتأكد، نلغي التنسيق ونطبع تاني:

```cpp
double x = 12.3456;
cout << fixed << setprecision(2) << x << endl;
cout.unsetf(ios::floatfield);
cout << setprecision(6) << x << endl;
```

الخرج

```
12.35
12.3456
```

`cout.unsetf(ios::floatfield)` بتلغي `fixed` ، و `setprecision(6)` بترجع الدقة الافتراضية، فظهرت تاني نفس القيمة اللي بدأنا بيها، `12.3456` . يعني القيمة المخزنة ما اتغيرتش، اللي اتغير كان شكل العرض بس.

نرجع لرقمنا

دلوقتي نطبق ده على أول سطر في الجدول:

```cpp
int mm = 2;
cout << fixed << setprecision(3) << mm / 25.4 << endl;
```

الخرج

```
0.079
```

0.0787... اتقرّبت لـ `0.079` ، لأن الرقم الرابع بعد العلامة 7 فالتالت طلع لفوق. وزي ما شفنا، ده عرض بس:

```
double inches = 2 / 25.4;
cout << fixed << setprecision(3) << inches << endl;
cout.unsetf(ios::floatfield);
cout << setprecision(12) << inches << endl;
```

الخرج

```
0.079
0.0787401574803
```

نكرر بالحلقة

دلوقتي نخلّي الحلقة تجيب كل المليمترات. هنجرب على أول خمس سطور بس (لحد 10) علشان الخرج يبقى قصير، ومن غير عرض خانات لسه:

```
for (int mm = 2; mm <= 10; mm = mm + 2)
{
    double inches = mm / 25.4;
    cout << mm << " " << fixed << setprecision(3) << inches << endl;
}
```

الخرج

```
2 0.079
4 0.157
6 0.236
8 0.315
10 0.394
```

الحلقة فيها تلات أجزاء: `int mm = 2` بتبدأ العداد، و `mm <= 10` بتتسأل قبل كل لفة، و `mm = mm + 2` بتتنفذ بعد كل لفة. علشان كده العداد مشي 2، 4، 6، 8، 10 وبعدين بقى 12 فالشرط بقى غلط والحلقة وقفت.

الأعمدة مش متساوية: الـ `10` زقت البوصات خطوة لليمين. نحط كل رقم في خانة عرضها 15 ناحية الشمال، ونحط `|` في آخر السطر علشان نشوف المسافات:

```
for (int mm = 2; mm <= 10; mm = mm + 2)
{
    double inches = mm / 25.4;
    cout << setw(15) << left << mm
         << setw(15) << left << fixed << setprecision(3) << inches
         << "|" << endl;
}
```

الخرج

```
2               0.079          |
4               0.157          |
6               0.236          |
8               0.315          |
10              0.394          |
```

كل سطر بقى بنفس الطول بالظبط، والعمود التاني بيبدأ من نفس المكان سواء المليمتر رقم واحد ولا رقمين. ولاحظوا إن `setw(15)` متكررة قبل كل خانة، لأنها بتخلص بعد حاجة واحدة.

البرنامج كامل

البرنامج الكامل هو نفس الحلقة، بس لحد 98، ومعاها عنوان للأعمدة وخط تحته. القيمة 25.4 متخزنة في ثابت `const` اسمه واضح، بدل ما تبقى رقم مكتوب جوه المعادلة.

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double MM_PER_INCH = 25.4;

    // 1. print the column headings
    cout << setw(15) << left << "Millimeters"
         << setw(15) << left << "Inches" << endl;
    cout << "----------------------" << endl;

    // 2. convert each requested millimeter value
    for (int mm = 2; mm <= 98; mm=mm+2) {
        double inches = mm / MM_PER_INCH;
        cout << setw(15) << left << mm
             << setw(15) << left << fixed << setprecision(3) << inches << endl;
    }
    return 0;
}
```

الخرج

```
Millimeters    Inches
-----------------------
2              0.079
4              0.157
6              0.236
8              0.315
10             0.394
12             0.472
14             0.551
16             0.630
18             0.709
20             0.787
22             0.866
24             0.945
26             1.024
28             1.102
30             1.181
32             1.260
34             1.339
36             1.417
38             1.496
40             1.575
42             1.654
44             1.732
46             1.811
48             1.890
50             1.969
52             2.047
54             2.126
56             2.205
58             2.283
60             2.362
62             2.441
64             2.520
66             2.598
68             2.677
70             2.756
72             2.835
74             2.913
76             2.992
78             3.071
80             3.150
82             3.228
84             3.307
86             3.386
88             3.465
90             3.543
92             3.622
94             3.701
96             3.780
98             3.858
```

حل تاني: الضرب في المقلوب

القسمة على 25.4 هي هي الضرب في 25.4 ÷ 1. البرنامج البديل بيغير سطر الحساب بس:

```
double inches = mm * (1.0 / MM_PER_INCH);
```

`1.0 / MM_PER_INCH` بتحسب عدد البوصات في المليمتر الواحد، وبعدين بنضرب فيه عدد المليمترات. في الرياضة الطريقتين واحد. في الكمبيوتر، الحسابات العشرية ممكن تختلف في آخر رقم بعيد جداً حسب ترتيب العمليات، فقارنا الخرج سطر سطر: التسعة وأربعين سطر طالعين هما هما بالظبط بالتلات أرقام اللي بنعرضها.

البرنامج البديل كامل

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double MM_PER_INCH = 25.4;

    // 1. print the column headings
    cout << setw(15) << left << "Millimeters"
         << setw(15) << left << "Inches" << endl;
    cout << "----------------------" << endl;

    // 2. convert each requested millimeter value
    for (int mm = 2; mm <= 98; mm=mm+2) {
        double inches = mm * (1.0 / MM_PER_INCH);
        cout << setw(15) << left << mm
             << setw(15) << left << fixed << setprecision(3) << inches << endl;
    }
    return 0;
}
```

ممكن تقع في ده

- **تضرب بدل ما تقسم.**

غلط مقصود: ضرب بدل قسمة

```
int mm = 2;
cout << mm * 25.4 << endl;
```

الخرج

```
50.8
```

2 مليمتر مستحيل يبقوا 50.8 بوصة. لو الرقم طلع أكبر من المليمترات، الاتجاه غلط.

- **تكتب الثابت رقم صحيح.** لو الطرفين `int` ، القسمة بتبقى قسمة أعداد صحيحة والكسر بيتشال:

غلط مقصود: الطرفين أعداد صحيحة

```
int mm = 2;
cout << mm / 25 << endl;
```

الخرج

```
0
```

- **تنسى `fixed` .** `setprecision(3)` لوحدها معناها تلات أرقام معنوية. ومع الرقم ده بالذات (`0.0787…` ، اللي بيبدأ بصفر بعد العلامة) ده طلّع 4 أرقام بعد العلامة:

غلط للجدول ده: ناقص `fixed`

```
cout << setprecision(3) << 2 / 25.4 << endl;
```

الخرج

```
0.0787
```

- **تكتب `>` بدل `=>` .** آخر سطر هيختفي:

غلط مقصود: `>` بدل `=>`

```
int last = 0;
for (int mm = 2; mm < 98; mm = mm + 2)
    last = mm;
cout << "last mm = " << last << endl;
```

الخرج

```
last mm = 96
```

- **تكتب `mm++` .** العداد هيمشي 2، 3، 4، ... والجدول هيبقى فيه الأعداد الفردية كمان. المطلوب زيادة 2.
- **تفتكر إن التقريب غيّر القيمة.** `0.079` شكل عرض بس. لو استخدمت المتغير في حساب تاني، هيستخدم القيمة المخزنة، مش الـ `0.079` المعروضة.

ملاحظة على الشيت

الأرقام اللي في جدول الشيت (`0.078` و `0.156` و `3.744` و `3.822`) مش هي اللي بتطلع من القسمة على 25.4. هي بتطابق الضرب في رقم مقرّب، 0.039 بوصة لكل ملليمتر:

للمقارنة: بالمعامل 0.039، مش 25.4 المكتوب في السؤال

```
cout << fixed << setprecision(3) << 2 * 0.039 << " " << 98 * 0.039 << endl;
```

الخرج

```
0.078 3.822
```

البرنامج هنا ماشي على المعلومة المكتوبة في السؤال (25.4)، فبيطلع `0.079` و `0.157` و `3.780` و `3.858` للسطور دي. وحل المصدر بيستخدم نفس القسمة بس بأربع أرقام بعد العلامة:

للمقارنة: بأربع أرقام زي حل المصدر

```
cout << fixed << setprecision(4) << 2 / 25.4 << " " << 98 / 25.4 << endl;
```

الخرج

```
0.0787 3.8583
```

اختارنا هنا تلات أرقام علشان تطابق شكل الجدول اللي في السؤال. عدد الأرقام بعد العلامة بيتحدد من المطلوب في كل سؤال، مش رقم ثابت لكل المسائل. والمعادلة نفسها (القسمة على 25.4) حاجة، وشكل عرض النتيجة حاجة تانية.

---

3 جدول تحويل من متر لقدم

(3) Write a program that displays the following table (note that $1\,meter = 3.280\,feet$):

Meters	Feet
1	3.280
2	6.560
...	
14	45.920
15	49.200

المطلوب

جدول بعمودين: أمتار في الشمال، ونفس الطول بالقدم (feet) في اليمين. الأمتار من 1 لحد 15، وبتزيد واحد كل سطر، يعني 15 سطر. والمعلومة المعطاة: المتر الواحد فيه 3.280 قدم. الجدول في الشيت بيعرض الأقدام بتلات أرقام بعد العلامة (`3.280` و `6.560` و `45.920` و `49.200`).

الخلفية

المرة دي العكس من السؤال اللي فات. المتر أكبر من القدم، والمتر الواحد فيه 3.280 قدم. فلو عندك 2 متر، يبقى عندك 3.280 قدم مرتين. ده ضرب:

`feet = meters * 3.280`

اقروا الوحدات تاني: قدم = (قدم لكل متر) × متر، المتر بيروح مع المتر. القاعدة اللي تمشوا بيها في أي تحويل: بصوا الوحدة اللي في مقام المعامل. لو هي نفس وحدة الرقم اللي معاك، اضربوا. لو المعامل مكتوب بالعكس (زي 25.4 ملليمتر لكل بوصة وانت معاك ملليمتر)، اقسموا. والتأكيد بالعقل: عدد الأقدام لازم يطلع أكبر من عدد الأمتار، لأن القدم وحدة أصغر.

من C++ هنحتاج

ضرب `int` في رقم عشري، و`fixed` مع `setprecision(3)` علشان الأصفار اللي في الآخر تظهر، وحلقة. هنكتبها بـ `for` وبـ `while` ونشوف إن الاتنين نفس الأجزاء التلاتة في أماكن مختلفة.

نبني الحل خطوة خطوة

> كل كود صغير في الجزء ده بيتجرب لوحده: حطوه مكان السطر `// one short example here` جوه `main` في الإطار اللي في أول الصفحة، في برنامج جديد لكل مثال. ولو تحت الكود فيه «الإدخال»، اكتبوه لما البرنامج يستناكم.

سطر واحد

```cpp
int M = 1;
double Feet = M * 3.280;
cout << Feet << endl;
```

الخرج

```
3.28
```

الحساب صح، بس فين الصفر اللي في الشيت؟ العرض الافتراضي لـ `cout` بيعرض لحد ستة أرقام معنوية بس، ومش بيكمّل بأصفار: الأصفار اللي في آخر الكسر بتتشال لأنها ما بتغيرش قيمة الرقم. 3.28 و 3.280 نفس الرقم بالظبط.

نغير حاجة واحدة: نطلب تلات أرقام بعد العلامة.

```cpp
int M = 1;
double Feet = M * 3.280;
cout << fixed << setprecision(3) << Feet << endl;
```

الخرج

```
3.280
```

`fixed` مع `setprecision(3)` بتطلب تلات أرقام بالظبط، فالصفر رجع. ولو طبعنا رقمين ورا بعض، التنسيق بيفضل شغال على التاني من غير ما نكتبه تاني:

```
cout << fixed << setprecision(3) << 1 * 3.280 << " " << 2 * 3.280 << endl;
```

الخرج

```
3.280 6.560
```

الأجزاء التلاتة في حلقة العدّ بـ `for` و `while`

حلقة العدّ اللي بنكتبها هنا بـ `for` أو `while` محتاجة تلات حاجات: قيمة البداية (initialization)، وشرط بيتسأل قبل كل لفة (test)، وخطوة بتغير العداد بعد كل لفة (update). (الـ `do-while` اللي في السؤال 1 مختلفة: بتسأل الشرط بعد اللفة.) في `while` كل واحدة في سطر لوحدها، فسهل تشوفوهم:

```
int M = 1;           // initialization
while (M <= 3)       // test
{
    cout << M << " ";
    M++;             // update
}
cout << endl << "after the loop M = " << M << endl;
```

الخرج

```
1 2 3
after the loop M = 4
```

العداد بدأ بـ 1، واتطبع 1 و 2 و 3. بعد ما طبع 3، الـ `M++` خلته 4، والشرط `4 <= 3` غلط، فالحلقة وقفت. علشان كده بعد الحلقة `M` قيمته 4 مش 3: الحلقة بتقف لما العداد يعدّي آخر قيمة، مش وهو عليها.

في `for` الأجزاء التلاتة دي هي هي، بس متجمعة في سطر واحد بين قوسين ومفصولة بـ `;`:

```
for (int M = 1; M <= 15; M++)
```

البداية `int M = 1`، والشرط `M <= 15`، والخطوة `M++`. وفرق صغير: المتغير اللي بيتعرّف جوه أقواس `for` بيعيش جوه الحلقة بس، وما ينفعش تستخدمه بعدها.

البرنامج كامل

نحط سطر الحساب وسطر الطباعة جوه `for` من 1 لـ 15، ونستخدم `setw(10)` و `left` للأعمدة زي ما عملنا في السؤال 2.

```cpp
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double M_Per_Feet = 3.280;

    // 1. print the column headings
    cout << setw(10) << left << "meters"
         << setw(10) << left << "Feet" << endl;
    cout << "----------------" << endl;

    // 2. convert each requested meter value
    for (int M = 1; M <= 15; M++) {
        double Feet = M * M_Per_Feet;
        cout << setw(10) << left << M
             << setw(10) << left << fixed << setprecision(3) << Feet << endl;
    }
    return 0;
}
```

الخرج

```
meters    Feet
----------------
1         3.280
2         6.560
3         9.840
4         13.120
5         16.400
6         19.680
7         22.960
8         26.240
9         29.520
10        32.800
11        36.080
12        39.360
13        42.640
14        45.920
15        49.200
```

بصوا على العمود اليمين: كل الأرقام بتلات أرقام بعد العلامة، حتى `49.200` اللي قيمتها 49.2. ده من `fixed` ، مش من الحساب.

حل تاني: `while`

البرنامج البديل بيوزع أجزاء الحلقة التلاتة على سطور منفصلة:

```
int M = 1;
while (M <= 15) {
    double Feet = M * M_Per_Feet;
    cout << setw(10) << left << M
         << setw(10) << left << fixed << setprecision(3) << Feet << endl;
    M++;
}
```

البداية بقت قبل الحلقة، والشرط بين قوسين `while` ، والخطوة بقت آخر سطر جوه الحلقة. الخرج طالع هو هو، الـ 15 سطر نفس الحروف بالظبط. ولما يبقى عدد اللفات معروف من الأول زي هنا، `for` بتبقى أوضح لأن الأجزاء التلاتة قدامك في سطر واحد. `while` بتبقى أنسب لما ما تبقاش عارف هتلف كام مرة، زي قراية لحد الصفر في السؤال 1.

البرنامج البديل كامل

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double M_Per_Feet = 3.280;

    // 1. print the column headings
    cout << setw(10) << left << "meters"
         << setw(10) << left << "Feet" << endl;
    cout << "----------------" << endl;

    // 2. convert each requested meter value
    int M = 1;
    while (M <= 15) {
        double Feet = M * M_Per_Feet;
        cout << setw(10) << left << M
             << setw(10) << left << fixed << setprecision(3) << Feet << endl;
        M++;
    }
    return 0;
}
```

ممكن تقع في ده

- **تقسم بدل ما تضرب.** متر واحد مستحيل يبقى أقل من قدم:

غلط مقصود: قسمة بدل ضرب

```
cout << fixed << setprecision(3) << 1 / 3.280 << endl;
```

الخرج

```
0.305
```

- **تنسى `M++` في `while`.** العداد هيفضل 1 والشرط هيفضل صح، والحلقة مش هتقف أبداً. في `for` صعب تنساها لأن ليها مكان في السطر.
- **تحط `M++` قبل الطباعة.** في `while` ترتيب السطور مهم. هنا الشرط لحد 3، والزيادة قبل الطباعة:

غلط مقصود: الزيادة قبل الطباعة

```
int M = 1;
while (M <= 3)
{
    M++;
    cout << M << " ";
}
cout << endl;
```

الخرج

```
2 3 4
```

أول قيمة اتبلعت، وآخر قيمة عدّت الحد. الجدول الكامل هيتزحزح بنفس الشكل: هيفوّت أول متر، وهيطبع متر زيادة بعد آخر قيمة مطلوبة.

- **تنسى `fixed`.** هتطلع `3.28` مكان `3.280`، والعمود مش هيبقى بشكل الشيت.
- **تتلخبط من اسم الثابت.** في البرنامج الثابت اسمه `M_Per_Feet`، لكن قيمته 3.280 قدم لكل متر. الحساب صح لأنه بيضرب، بس الاسم ممكن يوهمك. اقرا القيمة ووحدتها، مش الاسم بس.

ملاحظة على حل المصدر

حل المصدر بيعمل نفس الحساب ونفس المدى، لكن بيستخدم `setprecision(2)`، فبيعرض رقمين بعد العلامة:

للمقارنة: رقمين بعد العلامة زي حل المصدر

```
cout << fixed << setprecision(2) << 1 * 3.280 << " " << 2 * 3.280 << endl;
```

الخرج

```
3.28 6.56
```

جدول الشيت فيه تلات أرقام (`3.280`)، فالبرنامج هنا استخدم `setprecision(3)`. الحساب واحد في الحالتين، والفرق في العرض بس.

4 جدولين جنب بعض: ملليمتر وبوصة

(4) Write a program that displays the following tables side by side (note that $1\ inch = 25.4\ millimeters$):

Millimeters	Inches		Inches	Millimeters
2	0.078	|	1	25.641
4	0.156	|	2	51.282
...				
98	3.822	|	49	1256.410
100	3.900	|	50	1282.051

المطلوب

جدولين في نفس السطور، بينهم خط `|`. الجدول الشمال بيحوّل من ملليمتر لبوصة: الملليمترات 2، 4، ...، 98، 100 (زوجي من 2 لـ 100). الجدول اليمين بيحوّل من بوصة لملليمتر: البوصات 1، 2، ...، 49، 50. كل جدول فيه 50 قيمة، فالاتنين بيخلصوا مع بعض في 50 سطر.

أهم حاجة تاخدوا بالكم منها: الجدول اليمين مش بيكمّل على الشمال. البوصة اللي في اليمين (1، 2، 3...) هي إدخال جديد خالص، مش نتيجة الشمال (0.079، 0.157...). الجدولين بس متحطوطين جنب بعض.

الخلفية

عندنا اتجاهين لنفس المعلومة (البوصة فيها 25.4 ملليمتر):

- من ملليمتر لبوصة: نقسم على 25.4، زي السؤال 2.
- من بوصة لملليمتر: نضرب في 25.4، لأن كل بوصة فيها 25.4 ملليمتر.

حل المصدر بيكتب الاتجاه التاني بطريقة مختلفة شوية، فلازم نفهم «المقلوب» (reciprocal). مقلوب 25.4 هو 1 ÷ 25.4، ومعناه كام بوصة في الملليمتر الواحد. والقسمة على المقلوب هي هي الضرب في الرقم الأصلي: لو قسمت على (1 ÷ 25.4) كأنك ضربت في 25.4.

من C++ هنحتاج

حلقة واحدة عدادها بيطلّع قيمتين، وقسمة أعداد صحيحة `j / 2` مقصودة، وثوابت عشرية، و`setw` على كل خانة لوحدها، ونص ثابت `" | "` بين الجدولين.

نبني الحل خطوة خطوة

> كل كود صغير في الجزء ده بيتجرب لوحده: حطوه مكان السطر `// one short example here` جوه `main` في الإطار اللي في أول الصفحة، في برنامج جديد لكل مثال. ولو تحت الكود فيه «الإدخال»، اكتبوه لما البرنامج يستناكم.

سطر واحد فيه أربع قيم

قبل أي حلقة، نكتب أول سطر بإيدينا. فيه أربع قيم ليها أسماء: `mms` و `Inches` في الشمال، و `inch` و `milli` في اليمين.

```
int j = 2;
int mms = j, inch = j / 2;
double Inches = mms / 25.4;
double milli = inch / (1 / 25.4);
cout << fixed << setprecision(3);
cout << mms << " " << Inches << " | " << inch << " " << milli << endl;
```

الخرج

```
2 0.079 | 1 25.400
```

الشمال: 2 ملليمتر مقسومة على 25.4، والنتيجة بتظهر `0.079` بوصة بعد التقريب لتلات أرقام بعد العلامة؛ القيمة المخزنة أطول من كده، زي ما شفنا في السؤال 2. اليمين: 1 بوصة مقسومة على المقلوب (25.4 ÷ 1)، والنتيجة بتظهر `25.400` ملليمتر. لاحظوا إن `fixed` و `setprecision(3)` اتكتبوا مرة واحدة واشتغلوا على الرقمين العشريين، وإن الـ `2` والـ `1` اتطبعوا من غير كسر لأنهم `int`.

عداد واحد يطلّع قيمتين

في السطر الأول الشمال 2 واليمين 1، في التاني 4 و 2، وهكذا. يعني اليمين دايماً نص الشمال. فالبرنامج بيلف بعداد واحد `j` من 2 لـ 100 بخطوة 2، وبياخد `mms = j` و `inch = j / 2`:

```
for (int j = 2; j <= 8; j = j + 2)
    cout << j << " -> " << j / 2 << endl;
```

الخرج

```
2 -> 1
4 -> 2
6 -> 3
8 -> 4
```

`j / 2` هنا قسمة أعداد صحيحة، لأن الاتنين `int`. القسمة دي بتشيل أي كسر:

توضيح بعدد فردي، مش من قيم الجدول

```
cout << 7 / 2 << " " << 7 / 2.0 << endl;
```

الخرج

```
3 3.5
```

`7 / 2` طلعت `3` مش 3.5. بس في برنامجنا `j` دايماً زوجي، فالقسمة على 2 مظبوطة وما فيش كسر يضيع. القسمة الصحيحة هنا مقصودة ومأمونة، علشان عايزين `inch` يفضل رقم صحيح.

المقلوب في الكود

السطر التاني غلط مقصود: `1 / 25` من غير علامة عشرية

```
cout << 1 / 25.4 << endl;
cout << 1 / 25 << endl;
```

الخرج

```
0.0393701
0
```

`1 / 25.4` طلعت `0.0393701` بوصة لكل ملليمتر. لكن `1 / 25` طلعت صفر، لأن الاتنين `int` . علشان كده البرنامج بيكتب `1/25.4` بالعلامة العشرية، فيبقى الثابت `Inch_Per_mm` رقم عشري صح. وبعدين `inch / Inch_Per_mm` بيرجعنا للضرب في 25.4.

الخانات والفاصل

كل قيمة من الأربعة ليها `setw(12)` خاص بيها، وبين الجدولين `" | "` . في العنوان حصلت حاجة تستاهل نبص عليها: البرنامج كاتب `"Inches |"` كنص واحد جوه `setw(12)` . النص ده طوله 13، أطول من الخانة:

```
cout << "[" << setw(12) << left << "Inches" << "]" << endl;
cout << "[" << setw(12) << left << "Inches      |" << "]" << endl;
```

الخرج

```
[Inches      ]
[Inches      |]
```

الكلمة القصيرة اتكملت لحد 12. النص الطويل اتطبع زي ما هو من غير ما يتقص ومن غير مسافة زيادة، لأن `setw` أقل عرض بس، زي ما شفنا في السؤال 2. علشان كده في الخرج تحت هتلاقوا العنوان `Inches|` لازق، والسطور `1 |` فيها مسافة.

البرنامج كامل

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double mm_Per_Inch = 25.4;
    const double Inch_Per_mm = 1/25.4;

    // 1. print the column headings
    cout << setw(12) << left << "mms"
         << setw(12) << left << "Inches      |";
    cout << setw(12) << left << "Inches"
         << setw(12) << left << "mms" << endl;
    cout << "-----------------------------------------------" << endl;

    int mms, inch;
    double Inches, milli;
    // 2. calculate both conversions for the same output row
    for (int j = 2; j <= 100; j=j+2) {
        mms = j;
        inch = j/2;
        Inches = mms / mm_Per_Inch;
        milli = inch / Inch_Per_mm;

        cout << setw(12) << left << mms
             << setw(12) << left << fixed << setprecision(3) << Inches << "| ";

        cout << setw(12) << left << inch
             << setw(12) << left << fixed << setprecision(3) << milli << endl;
    }

    return 0;
}
```

الخرج

```
mms          Inches      |Inches      mms
-------------------------------------------------
2            0.079       | 1          25.400
4            0.157       | 2          50.800
6            0.236       | 3          76.200
8            0.315       | 4          101.600
10           0.394       | 5          127.000
12           0.472       | 6          152.400
14           0.551       | 7          177.800
16           0.630       | 8          203.200
18           0.709       | 9          228.600
20           0.787       | 10         254.000
22           0.866       | 11         279.400
24           0.945       | 12         304.800
26           1.024       | 13         330.200
28           1.102       | 14         355.600
30           1.181       | 15         381.000
32           1.260       | 16         406.400
34           1.339       | 17         431.800
36           1.417       | 18         457.200
38           1.496       | 19         482.600
40           1.575       | 20         508.000
42           1.654       | 21         533.400
44           1.732       | 22         558.800
46           1.811       | 23         584.200
48           1.890       | 24         609.600
50           1.969       | 25         635.000
52           2.047       | 26         660.400
54           2.126       | 27         685.800
56           2.205       | 28         711.200
58           2.283       | 29         736.600
60           2.362       | 30         762.000
62           2.441       | 31         787.400
64           2.520       | 32         812.800
66           2.598       | 33         838.200
68           2.677       | 34         863.600
70           2.756       | 35         889.000
72           2.835       | 36         914.400
74           2.913       | 37         939.800
76           2.992       | 38         965.200
78           3.071       | 39         990.600
80           3.150       | 40         1016.000
82           3.228       | 41         1041.400
84           3.307       | 42         1066.800
86           3.386       | 43         1092.200
88           3.465       | 44         1117.600
90           3.543       | 45         1143.000
92           3.622       | 46         1168.400
94           3.701       | 47         1193.800
96           3.780       | 48         1219.200
98           3.858       | 49         1244.600
100          3.937       | 50         1270.000
```

حل تاني: نضرب على طول

بدل القسمة على المقلوب، البرنامج البديل بيضرب البوصات في 25.4 مباشرة، وبيستغنى عن الثابت التاني:

```
milli = inch * mm_Per_Inch;
```

الطريقة دي أقرب لمعنى التحويل («كل بوصة فيها 25.4 ملليمتر»). قارنا الخمسين سطر بتوع البديل بخرج البرنامج الأصلي، وطالعين هما هما بالظبط.

البرنامج البديل كامل

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double mm_Per_Inch = 25.4;

    // 1. print the column headings
    cout << setw(12) << left << "mms"
         << setw(12) << left << "Inches      |";
    cout << setw(12) << left << "Inches"
         << setw(12) << left << "mms" << endl;
    cout << "-----------------------------------------------" << endl;

    int mms, inch;
    double Inches, milli;
    // 2. calculate both conversions for the same output row
    for (int j = 2; j <= 100; j=j+2) {
        mms = j;
        inch = j/2;
        Inches = mms / mm_Per_Inch;
        milli = inch * mm_Per_Inch;

        cout << setw(12) << left << mms
             << setw(12) << left << fixed << setprecision(3) << Inches << "| ";

        cout << setw(12) << left << inch
             << setw(12) << left << fixed << setprecision(3) << milli << endl;
    }

    return 0;
}
```

ممكن تقع في ده

- **تاخد نتيجة الشمال وتحولها تاني.** لو حطيت `Inches` في الجدول اليمين، هترجع للملليمترات اللي بدأت بيها. اليمين ليه إدخال خاص بيه: 1، 2، 3، ...، 50.

- **تكتب `1/25` من غير علامة عشرية.** قسمة `int` على `int` وأصغر من واحد، يعني صفر. ومن غير ما تاخد بالك هتقسم على صفر بعدها.
- **تستخدم `j / 2` وعدادك بيمشي واحد واحد.** لو `j` بقى فردي، الكسر هيضيع وهيتكرر نفس رقم البوصة في سطرين.
- **تكتب `setw` مرة واحدة في أول السطر.** هتأثر على أول قيمة بس، والباقي هيلزق في بعضه.
- **تحط النص `" | "` جوه `setw`.** هيتعامل كخانة وهيتكمل مسافات، والفاصل هيبعد عن مكانه.

ملاحظة على الشيت

زي السؤال 2، أرقام جدول الشيت مش طالعة من 25.4. الشيت عارض `25.641` و `51.282` و `1256.410` و `1282.051` في العمود اليمين، و `3.900` قصاد 100 ملليمتر. الأرقام دي بتطابق استخدام 0.039 بوصة لكل ملليمتر بدل 25.4:

للمقارنة: بالمعامل 0.039، مش 25.4 المكتوب في السؤال

```
cout << fixed << setprecision(3) << 1 / 0.039 << " " << 100 * 0.039 << endl;
```

الخرج

```
25.641 3.900
```

البرنامج هنا ماشي على المعلومة المكتوبة في السؤال وعلى طريقة حل المصدر، فبيطلع `25.400` و `50.800` و `1244.600` و `1270.000` ، و `3.937` قصاد 100 ملليمتر. لو قارنتوا خرجكم بالشيت ولقيتوا الفرق ده، ده سببه.

---

5 جدولين جنب بعض: متر وقدم

(5) Write a program that displays the following tables side by side (note that $1\ meter = 3.280\ feet$):

Meters	Feet		Feet	Meters
1	3.280	|	3	0.915
2	6.560	|	6	1.829
...				
14	45.920	|	42	12.805
15	49.200	|	45	13.720

المطلوب

نفس فكرة السؤال 4 بوحدات تانية. الجدول الشمال: أمتار من 1 لـ 15 بزيادة 1، وجنبها الطول بالقدم. الجدول اليمين: أقدام من 3 لـ 45 بزيادة 3 (3، 6، 9، ...، 42، 45)، وجنبها الطول بالمتر. الاتنين 15 سطر. والمعلومة: المتر فيه 3.280 قدم. والجدول بيعرض الأرقام العشرية بتلات أرقام بعد العلامة.

وتاني: الجدول اليمين ليه إدخال خاص بيه (3، 6، 9...)، مش نتيجة الشمال (3.280، 6.560...).

الخلفية

- من متر لقدم: نضرب في 3.28، زي السؤال 3. المتر فيه 3.28 قدم.
- من قدم لمتر: نقسم على 3.28. السؤال هنا «كام مرة 3.28 قدم داخلة في عدد الأقدام اللي معايا؟».

والتأكيد بالعقل في الاتجاه التاني: عدد الأمتار لازم يطلع أصغر من عدد الأقدام. 3 قدم أقل من متر.

من C++ هنحتاج

نفس أدوات السؤال 4: عداد واحد بيطلّع قيمتين، وثوابت عشرية، وتنسيق بـ `fixed` و `setprecision(3)` و `setw`. الجديد إن القيمة التانية مش نص العداد، دي تلات أمثاله، يعني العداد مضروب في 3.

نبني الحل خطوة خطوة

> كل كود صغير في الجزء ده بيتجرب لوحده: حطوه مكان السطر `// one short example here` جوه `main` في الإطار اللي في أول الصفحة، في برنامج جديد لكل مثال. ولو تحت الكود فيه «الإدخال»، اكتبوه لما البرنامج يستناكم.

عداد واحد وعمودين بخطوتين مختلفتين

العمود الشمال بيزيد 1 والعمود اليمين بيزيد 3. لو العداد `j` بيمشي 1، 2، 3...، يبقى الشمال هو `j` واليمين هو `3 * j`:

```
for (int j = 1; j <= 4; j++)
    cout << j << "  " << 3 * j << endl;
```

الخرج

```
1  3
2  6
3  9
4  12
```

الحلقة لفت مرة واحدة لكل سطر، وكل لفة طلّعت الرقمين اللي في نفس السطر. مش محتاجين عداد تاني بيزيد 3.

سطر واحد كامل

```
int j = 1;
int Meters = j, Feet = 3 * j;
double Ft = Meters / (1 / 3.28);
double Met = Feet / 3.28;
cout << fixed << setprecision(3);
cout << Meters << " " << Ft << " | " << Feet << " " << Met << endl;
```

الخرج

```
1 3.280 | 3 0.915
```

الشمال: 1 متر بيظهر `3.280` قدم. اليمين: 3 قدم مقسومة على 3.28، والنتيجة بتظهر `0.915` متر بعد التقريب لتلات أرقام بعد العلامة؛ الكسر الحقيقي أطول من كده. لاحظوا إن حل المصدر بيحسب الشمال بالقسمة على المقلوب:
`Meters / meter_Per_feet`، و`meter_Per_feet` قيمتها `1/3.28`. القسمة على (3.28 ÷ 1) هي الضرب في 3.28، زي ما شفنا في السؤال 4.

3.28 ولا 3.280؟

السؤال كاتب 3.280، والبرنامج كاتب `3.28`. فيه فرق؟

```
cout << 3.28 << " " << 3.280 << endl;
cout << (3.28 == 3.280) << endl;
```

الخرج

```
3.28 3.28
1
```

نفس الرقم بالظبط، والمقارنة `==` بتقول صح (`1`). الصفر الأخير ملوش قيمة في الحساب. اللي بيخليه يظهر في الجدول هو `fixed` مع `setprecision(3)`، يعني قرار عرض مش قرار حساب.

البرنامج كامل

```cpp
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double feet_Per_meter = 3.28;
    const double meter_Per_feet = 1/3.28;

    // 1. print the column headings
    cout << setw(12) << left << "Meters"
         << setw(12) << left << "Feet        |";
    cout << setw(12) << left << "Feet"
         << setw(12) << left << "Meters" << endl;
    cout << "-----------------------------------------------" << endl;

    int Meters, Feet;
    double Met, Ft;
    // 2. calculate both conversions for the same output row
    for (int j = 1; j <= 15; j++) {
        Meters = j;
        Feet = 3*j;
        Met = Feet / feet_Per_meter;
        Ft = Meters / meter_Per_feet;

        cout << setw(12) << left << Meters
             << setw(12) << left << fixed << setprecision(3) << Ft << "| ";

        cout << setw(12) << left << Feet
             << setw(12) << left << fixed << setprecision(3) << Met << endl;
    }

    return 0;
}
```

الخرج

```
Meters      Feet        |Feet        Meters
-----------------------------------------------
1           3.280       | 3           0.915
2           6.560       | 6           1.829
3           9.840       | 9           2.744
4           13.120      | 12          3.659
5           16.400      | 15          4.573
6           19.680      | 18          5.488
7           22.960      | 21          6.402
8           26.240      | 24          7.317
9           29.520      | 27          8.232
10          32.800      | 30          9.146
11          36.080      | 33          10.061
12          39.360      | 36          10.976
13          42.640      | 39          11.890
14          45.920      | 42          12.805
15          49.200      | 45          13.720
```

حل تاني: نضرب على طول

البديل بيحسب الأقدام بالضرب المباشر وبيشيل ثابت المقلوب:

```
Ft = Meters * feet_Per_meter;
```

الخمستاشر سطر طالعين هما هما في البرنامجين. الطريقة دي بتقرا زي جملة التحويل نفسها: «الأمتار في 3.28».

البرنامج البديل كامل

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double feet_Per_meter = 3.28;

    // 1. print the column headings
    cout << setw(12) << left << "Meters"
         << setw(12) << left << "Feet        |";
    cout << setw(12) << left << "Feet"
         << setw(12) << left << "Meters" << endl;
    cout << "-----------------------------------------------" << endl;

    int Meters, Feet;
    double Met, Ft;
    // 2. calculate both conversions for the same output row
    for (int j = 1; j <= 15; j++) {
        Meters = j;
        Feet = 3*j;
        Met = Feet / feet_Per_meter;
        Ft = Meters * feet_Per_meter;

        cout << setw(12) << left << Meters
             << setw(12) << left << fixed << setprecision(3) << Ft << "| ";

        cout << setw(12) << left << Feet
             << setw(12) << left << fixed << setprecision(3) << Met << endl;
    }

    return 0;
}
```

ممكن تقع في ده

- **تحوّل نتيجة الشمال تاني.** لو أخدت الأقدام اللي طلعت في الشمال وقسمتها على 3.28، هترجع للمتر اللي بدأت بيه:

غلط مقصود: حولنا نتيجة الشمال تاني

```
double Ft = 1 * 3.28;
cout << fixed << setprecision(3) << Ft / 3.28 << endl;
```

الخرج

```
1.000
```

ده مش المطلوب. اليمين بيبدأ من 3 قدم.

- **تعمل العمود اليمين `3 + j`.** ده بيزيد واحد بس كل سطر زي العداد نفسه، مجرد إنه مزحوح. المطلوب بيزيد 3 كل سطر، يعني `3 * j`.
- **تقسم الأمتار وتضرب الأقدام.** بص على النتيجة: لو 3 قدم طلعوا أكتر من 3 متر، الاتجاه اتعكس.
- **تفتكر إن `3.28` غلط لأن السؤال كاتب 3.280.** الاتنين رقم واحد. الأصفار اللي في الآخر بتظهر بالتنسيق بس.
- **تحط `setprecision` مختلف لكل عمود من غير ما تقصد.** التنسيق بيفضل شغال على `cout` لحد ما يتغير، فأي `setprecision` بتكتبها بتأثر على كل الأرقام العشرية اللي بعدها.

السؤال ده جدوله ماشي مع الحساب: `0.915` و`1.829` و`12.805` و`13.720` في الشيت هي نفس اللي طلع في التشغيل.

---

6 جدول tan و cot من 0 لـ 60 درجة

(6) Write a C++ program to print the following table to display the tan and cot values of degrees from 0 to 60 with increments of 10 degrees. Round the value to keep four digits after the decimal point.

Degree	Sin/Cos	Cos/Sin
10	0.0000	inf
20	0.1736	5.6713
...		
50	1.1918	0.8391
60	1.7320	0.5774

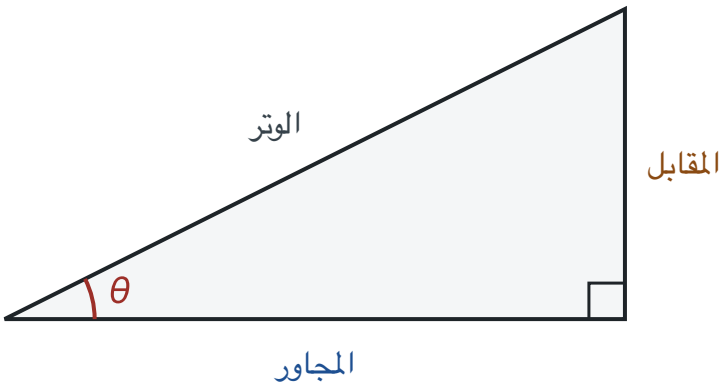
المطلوب

جدول فيه الزوايا 0، 10، 20، ...، 60 درجة (سبع زوايا، بزيادة 10)، وقصاد كل زاوية قيمة tan وقيمة cot، متقربين لأربع أرقام بعد العلامة العشرية. عناوين الأعمدة في الشيت `Sin/Cos` و`Cos/Sin`، ودول هما نفسهم tan و cot زي ما هنشوف.

وفيه نقطة لازم تتحسم من الأول: عند الزاوية صفر، cot مش موجودة (undefined)، فالبرنامج لازم يقول كده بدل ما يطبع رقم.

الخلفية

خدوا مثلث قايم الزاوية، واختاروا واحدة من الزاويتين الحادتين، ونسميها θ. الضلع اللي قصادها اسمه «المقابل»، والضلع اللي جنبها (غير الوتر) اسمه «المجاور»، والضلع الطويل قصاد الزاوية القايمة اسمه «الوتر».



الـ tan والـ cot نسبتين بين الضلعين دول:

- **المجاور ÷ المقابل = tan θ**
- **المقابل ÷ المجاور = cot θ**، يعني نفس النسبة مقلوبة، فـ cot هو مقلوب tan.

وفيه علاقة تانية بتشرح عناوين الشيت: الوتر ÷ المقابل = sin θ، و الوتر ÷ المجاور = cos θ. لو قسمت sin على cos، الوتر بيروح مع الوتر، ويفضل المجاور ÷ المقابل، اللي هو tan. علشان كده tan θ = sin θ / cos θ، و cot θ = cos θ / sin θ.

تخيلوا الزاوية θ بتصغر لحد ما توصل صفر. الضلع المقابل بيقصر لحد ما يبقى صفر. ساعتها tan بيبقى المجاور ÷ صفر، يعني صفر. أما cot فبيبقى صفر ÷ المجاور، والقسمة على صفر ملهاش قيمة. علشان كده cot عند الصفر undefined.

الدرجات والراديان

الزوايا ليها وحدتين. الدرجة (degree): اللفة الكاملة 360 درجة. والراديان (radian) بيقيس الزاوية بطول القوس اللي بتقطعه من دايرة، مقسوم على نص قطر الدايرة. لو الزاوية لفة كاملة، القوس هو المحيط كله، وطوله 2πr، ولما نقسمه على r يفضل 2π. يعني اللفة الكاملة 2π راديان، ونص اللفة (180 درجة) هو π راديان. ومن هنا قاعدة التحويل:

```
radians = degrees * PI / 180
```

والمهم هنا: دوال `<cmath>` زي `tan` و `sin` و `cos` بتفهم الزاوية بالراديان بس. لو اديتها 30 هتفهمها 30 راديان، مش 30 درجة.

من C++ هنحتاج

`<cmath>` علشان `tan` و `sin` و `cos` و `acos`، وقيمة π في ثابت، وحلقة بتزيد 10، و `if` علشان الزاوية صفر، و `fixed` مع `setprecision(4)`.

نبني الحل خطوة خطوة

> كل كود صغير في الجزء ده بيتجرب لوحده: حطوه مكان السطر `// one short example here` جوه `main` في الإطار اللي في أول الصفحة، في برنامج جديد لكل مثال. ولو تحت الكود فيه «الإدخال»، اكتبوه لما البرنامج يستناكم.

نجيب π منين

C++98 ما فيهاش ثابت قياسي لـ π. لكن إحنا عارفين إن cos π = −1، يعني الزاوية اللي الـ cos بتاعها −1 هي π. والدالة `acos` بتعمل العكس: تديها قيمة الـ cos وتديك الزاوية.

```
const double PI = acos(-1.0);
cout << PI << endl;
cout << fixed << setprecision(10) << PI << endl;
```

الخرج

```
3.14159
3.1415926536
```

السطر الأول بالعرض الافتراضي (لحد ستة أرقام معنوية، وهنا اتعرض بالستة)، والتاني بعشرة أرقام بعد العلامة علشان تشوفوا إن القيمة المخزنة أدق بكتير من اللي بيظهر.

ليه لازم نحوّل لراديان

السطر الأول غلط مقصود: `tan(30.0)` بيفهم 30 راديان

```
const double PI = acos(-1.0);
cout << tan(30.0) << endl;
cout << tan(30 * PI / 180) << endl;
```

الخرج

```
-6.40533
0.57735
```

السطر الأول اتحسب على إن الزاوية 30 راديان، ودي زاوية تانية خالص غير 30 درجة، فطلع رقم ملوش علاقة باللي عايزينه. السطر التاني بعد التحويل طلع `0.57735`، وده tan الـ 30 درجة.

سطر واحد من الجدول

```
const double PI = acos(-1.0);
int j = 30;
double rad = j * PI / 180;
cout << rad << endl;
cout << fixed << setprecision(4)
     << j << "  " << tan(rad) << "  " << 1 / tan(rad) << endl;
```

الخرج

```
0.523599
30  0.5774  1.7321
```

الـ 30 درجة بقت `0.523599` راديان. وبعدين `fixed` و `setprecision(4)` قرّبوا tan لـ `0.5774`، و cot (اللي هي 1 ÷ tan) لـ `1.7321`.

الزاوية صفر

نشوف C++ بتعمل إيه لو قسمنا على tan الصفر:

تجربة: ده اللي بيحصل لو قسمنا عند الصفر، والبرنامج الكامل مش بيعمل كده

```
cout << tan(0.0) << endl;
cout << 1 / tan(0.0) << endl;
```

الخرج

```
0
inf
```

tan الصفر طلع `0` زي ما توقعنا. والقسمة طلّعت `inf`، وده تمثيل جوه الكمبيوتر لنتيجة قسمة رقم عشري على صفر. مش معناه إن cot ليها قيمة، ومش مضمون تطلع بنفس الشكل على كل مترجم. علشان كده البرنامج بيسأل عن الزاوية نفسها قبل ما يقسم:

```
if (j == 0) cout << "undefined" << endl;
else cout << fixed << setprecision(4) << 1/tan(rad) << endl;
```

لاحظوا إن الشرط بيسأل عن `j`، وهو `int` قيمته صفر بالظبط، مش عن نتيجة `tan` اللي هي رقم عشري.

البرنامج كامل

```cpp
#include<iostream>
#include <iomanip>
#include <cmath>
using namespace std;

int main() {
    const double PI = acos(-1.0);
    double sin_per_cos, rad;

    // 1. print the column headings
    cout << setw(10) << left << "angle"
         << setw(10) << left << "sin/cos" << setw(10) << left << "cos/sin" << endl;

    // 2. convert each degree value to radians
    for(int j=0;j<=60;j=j+10)
    {
        rad=j*PI/180;
        sin_per_cos=tan(rad);
        cout << setw(10) << left << j
             << setw(10) << left << fixed << setprecision(4) << sin_per_cos;
        // 3. cot is defined only when tan is nonzero
        if (j == 0) cout << "undefined" << endl;
        else cout << fixed << setprecision(4) << 1/tan(rad) << endl;
    }
    return 0;
}
```

الخرج

```
angle     sin/cos   cos/sin
0         0.0000    undefined
10        0.1763    5.6713
20        0.3640    2.7475
30        0.5774    1.7321
40        0.8391    1.1918
50        1.1918    0.8391
60        1.7321    0.5774
```

بصوا على سطر 30 وسطر 60: $\tan 30° = \cot 60° = 0.5774$، و $\cot 30° = \tan 60° = 1.7321$. ده منطقي من المثلث: الزاويتين الحادتين في المثلث القايم مجموعهم 90، فالمقابل لواحدة هو المجاور للتانية.

حل تاني: sin/cos و cos/sin

البديل بيكتب النسبتين زي عناوين الأعمدة بالظبط، بدل `tan` و `1/tan`:

```cpp
sin_per_cos=sin(rad)/cos(rad);
```

```
else cout << fixed << setprecision(4) << cos(rad)/sin(rad) << endl;
```

الشرط بتاع الصفر لسه موجود، لأن sin الصفر صفر، والقسمة عليه نفس المشكلة. السبع سطور طالعين هما هما بالظبط في البرنامجين بالأربع أرقام اللي بنعرضها.

البرنامج البديل كامل

```
#include<iostream>
#include <iomanip>
#include <cmath>
using namespace std;

int main() {
    const double PI = acos(-1.0);
    double sin_per_cos, rad;

    // 1. print the column headings
    cout << setw(10) << left << "angle"
         << setw(10) << left << "sin/cos" << setw(10) << left << "cos/sin" << endl;

    // 2. convert each degree value to radians
    for(int j=0;j<=60;j=j+10)
    {
        rad=j*PI/180;
        sin_per_cos=sin(rad)/cos(rad);
        cout << setw(10) << left << j
             << setw(10) << left << fixed << setprecision(4) << sin_per_cos;
        // 3. cot is defined only when tan is nonzero
        if (j == 0) cout << "undefined" << endl;
        else cout << fixed << setprecision(4) << cos(rad)/sin(rad) << endl;
    }
    return 0;
}
```

ممكن تقع في ده

- **تدّي `tan` الدرجات على طول.** `tan(30.0)` طلعت `-6.40533` فوق. حوّل لراديان الأول.
- **ترتب التحويل غلط مع `int`.**

غلط مقصود: القسمة الصحيحة جت الأول

```
const double PI = acos(-1.0);
int j = 30;
double rad = j / 180 * PI;
cout << rad << "  " << tan(rad) << endl;
```

الخرج

```
0  0
```

`j / 180` اتحسبت الأول، والاتنين `int`، فـ 30 / 180 بقت صفر قبل ما نضرب في π. اكتبها `j * PI / 180` علشان الضرب في الرقم العشري ييجي الأول.

- **تستخدم `M_PI`.** بتشتغل على مترجمات كتير، لكنها مش جزء مضمون من C++98. `acos(-1.0)` مضمونة من `<cmath>`.
- **تطبع `inf` وتعتبره إجابة.** cot الصفر مش رقم كبير، هي مش معرّفة. قول كده صراحة.
- **تختبر `tan(rad) == 0`.** مقارنة الأرقام العشرية بالمساواة بتخون في حالات كتير. اسأل عن الزاوية الصحيحة `j == 0`.

ملاحظة على الشيت

الجدول المطبوع في الشيت فيه لخبطة في أول سطرين. السطر المكتوب جنبه 10 فيه قيم الزاوية صفر (`0.0000` و `inf`). والسطر المكتوب جنبه 20 فيه `0.1736` و `5.6713`، ودول sin الـ 10 درجة و cot الـ 10 درجة، مش tan و cot الـ 20:

للمقارنة: قيم الـ 10 درجة اللي ظاهرة في الشيت

```
const double PI = acos(-1.0);
double rad = 10 * PI / 180;
cout << fixed << setprecision(4)
     << sin(rad) << "  " << tan(rad) << "  " << 1 / tan(rad) << endl;
```

الخرج

```
0.1736  0.1763  5.6713
```

السطر ده فيه sin الـ 10، وبعده tan الـ 10، وبعده cot الـ 10. وفي آخر سطر الشيت كاتب 1.7320 = tan 60°، والحساب متقرب لأربع أرقام بيطلع `1.7321`. البرنامج هنا ماشي على نص السؤال: من 0 لـ 60 بزيادة 10، وكل سطر ليه قيم زاويته هو.

---

7 جدول الدالة exp من 0 لـ 10

(7) Write a C++ program to print the following table using the *exp* function:

Number	Exponent
0	1
1	2.71828
...	
9	8103.08
10	22026.5

المطلوب

جدول عمودين: `x` من 0 لـ 10 (أعداد صحيحة، بزيادة 1، يعني 11 سطر)، وقصاد كل واحد قيمة `exp(x)` . الشيت بيطلب صراحة استخدام الدالة `exp` . والأرقام في جدول الشيت (`1` و `2.71828` و `8103.08` و `22026.5`) معروضة بالشكل الافتراضي، من غير عدد ثابت من الأرقام بعد العلامة.

الخلفية

فيه رقم ثابت في الرياضة اسمه e، قيمته تقريباً 2.71828، زيه زي π: رقم كسره ما بيخلصش. و `exp(x)` معناها e أُس x. الأساس (e) ثابت، والأُس (x) هو اللي بيتغير. ولما x عدد صحيح موجب، زي 1 و 2 و 3 في الجدول ده، e أُس x معناها e مضروبة في نفسها x مرة. (الدالة نفسها شغالة مع أي رقم عشري كمان، لكن ده مش محتاجينه هنا.)

- `exp(0)` : $e^0 = 1$، لأن أي رقم غير الصفر أُس صفر بيساوي 1، و e مش صفر.
- `exp(1)` : $e^1 = e$ نفسها.
- `exp(2)` : $e^2 = e \times e$.

والقيم بتكبر بسرعة جداً، لأن كل خطوة في الجدول بتضرب في e تاني، مش بتزوّد رقم ثابت.

من C++ هنحتاج

الدالة `exp` من `<cmath>` ، وحلقة من 0 لـ 10، والعرض الافتراضي لـ `cout` (لحد ستة أرقام معنوية)، و `setw` مع `left` للأعمدة.

نبني الحل خطوة خطوة

> كل كود صغير في الجزء ده بيتجرب لوحده: حطوه مكان السطر `// one short example here` جوه `main` في الإطار اللي في أول الصفحة، في برنامج جديد لكل مثال. ولو تحت الكود فيه «الإدخال»، اكتبوه لما البرنامج يستناكم.

أول تلات قيم

```
cout << exp(0.0) << endl;
cout << exp(1.0) << endl;
cout << exp(2.0) << endl;
```

الخرج

```
1
2.71828
7.38906
```

`exp(0.0)` طلعت `1` بالظبط، و `exp(1.0)` طلعت e نفسها معروضة بستة أرقام معنوية (الكسر طويل، فاتعرض بالحد الأقصى)، و `exp(2.0)` طلعت e × e.

مش هي دي

فيه أربع حاجات شبه `exp(x)` في الشكل وبتطلّع أرقام تانية خالص. نجربهم كلهم على x = 2:

مقارنة: السطر الأول بس هو `exp(x)` المطلوب

```
double e = exp(1.0);
int x = 2;
cout << exp(x) << endl;
cout << x * e << endl;
cout << pow(2.0, e) << endl;
cout << (x ^ 3) << endl;
cout << pow(2.0, 3.0) << endl;
```

الخرج

```
7.38906
5.43656
6.58089
1
8
```

- `exp(x)` : e أُس 2، وطلعت `7.38906` . ده المطلوب.
- `x * e` : 2 مضروبة في e، وطلعت `5.43656` . ضرب، مش أُس.
- `pow(2.0, e)` : 2 أُس e، وطلعت `6.58089` . الأساس والأُس متبدلين.
- `x ^ 3` : طلعت `1` ! العلامة `^` في C++ مش أُس خالص. دي عملية تانية خالص على البتات اسمها XOR، مش موضوعنا هنا. المهم إنها مش أُس.
- `pow(2.0, 3.0)` : هي دي الطريقة الصح لأي أُس في C++، وطلعت `8` .

يعني لو عايز e أُس x، استخدم `exp(x)` . ولو عايز أي أساس تاني أُس حاجة، استخدم `pow` . والـ `^` ما تستخدمهاش للأُس أبداً في C++.

ليه مش هنستخدم `fixed` هنا

في الأسئلة اللي فاتت كان الجدول عايز عدد ثابت من الأرقام بعد العلامة. هنا لأ. خلونا نشوف آخر قيمة في الجدول بالعرض الافتراضي، وبعدين بدقة أكبر:

```
cout << exp(10.0) << endl;
cout << setprecision(8) << exp(10.0) << endl;
```

الخرج

```
22026.5
22026.466
```

`cout` من غير أي تنسيق بيعرض لحد ستة أرقام معنوية. الرقم ده كسره طويل، فاتعرض بالستة: `22026.5` (خمسة قبل العلامة وواحد بعدها). لما طلبنا 8 أرقام معنوية ظهر أكتر من الكسر. ده نفس الشكل اللي في جدول الشيت. لو حطينا `fixed` من نفسنا، الشكل هيتغير:

غلط للسؤال ده: `fixed` مش مطلوبة

```
cout << fixed << setprecision(2) << exp(0.0) << "  " << exp(10.0) << endl;
```

الخرج

```
1.00  22026.47
```

`1.00` و `22026.47` مش زي الشيت. السؤال ما طلبش عدد أرقام بعد العلامة، فبنسيب العرض الافتراضي.

البرنامج كامل

```
#include<iostream>
#include <iomanip>
#include <cmath>
using namespace std;

int main() {
    double Exponent;

    // 1. print the column headings
    cout << setw(10) << left << "Number(x)"
         << setw(10) << left << "Exponent(e^x)" << endl;

    // 2. calculate the exponential for each argument
    for(int j=0;j<=10;j++)
    {
        Exponent = exp(j);
        cout << setw(10) << left << j << setw(11) << left << Exponent <<endl;
    }
    return 0;
}
```

الخرج

```
Number(x) Exponent(e^x)
0         1
1         2.71828
2         7.38906
3         20.0855
4         54.5982
5         148.413
6         403.429
7         1096.63
8         2980.96
9         8103.08
10        22026.5
```

في الجدول ده كل الأرقام ظهرت بستة أرقام معنوية، لأن كسورها طويلة، ما عدا `1` اللي مش محتاجة غير رقم واحد. الستة حد أقصى للعرض الافتراضي، مش عدد بيتكمّل دايماً. ولاحظوا إن `exp(j)` بتاخد `j` وهو `int` ، وبيتحسب كرقم عشري. والعنوان `"Exponent(e^x)"` جوه علامات تنصيص، فالـ `^` فيه مجرد حرف بيتطبع، مش عملية.

حل تاني: `while`

```
int j = 0;
while (j <= 10)
{
    Exponent = exp(j);
    cout << setw(10) << left << j << setw(11) << left << Exponent <<endl;
    j++;
}
```

نفس الأجزاء التلاتة: البداية `j = 0` قبل الحلقة، والشرط `j <= 10` ، والزيادة في آخر الجسم. الـ 11 سطر طالعين هما هما في البرنامجين.

البرنامج البديل كامل

```cpp
#include<iostream>
#include <iomanip>
#include <cmath>
using namespace std;

int main() {
    double Exponent;

    // 1. print the column headings
    cout << setw(10) << left << "Number(x)"
         << setw(10) << left << "Exponent(e^x)" << endl;

    // 2. calculate the exponential for each argument
    int j = 0;
    while (j <= 10)
    {
        Exponent = exp(j);
        cout << setw(10) << left << j << setw(11) << left << Exponent <<endl;
        j++;
    }
    return 0;
}
```

ممكن تقع في ده

- **تكتب `e^x` أو `2.71828 ^ x`** . الـ `^` مش أُس في C++. ومع رقم عشري المترجم هيرفضها أصلاً.
- **تكتب `2.71828 * x`** . ده e مضروبة في x، مش e أُس x.
- **تكتب `pow(x, 2.71828)`** . ده x أُس e. الأساس لازم يبقى e.
- **تحط `fixed` من غير ما السؤال يطلبه.** الأرقام هتبقى صح لكن شكلها مش زي المطلوب.
- **توقف الحلقة عند `j < 10`** . السطر بتاع 10 هيختفي، والسؤال عايز من 0 لـ 10 الاتنين داخلين.
- **تنسى `#include <cmath>`** . `exp` و `pow` متعرفين فيها.

جدول السؤال ده ماشي مع التشغيل: `1` و `2.71828` و `8103.08` و `22026.5` هي نفس القيم اللي في الشيت.

---

8 من 1 لـ 500: يقبل القسمة على 3 و 4

(8) Write a C++ program that displays all the numbers from 1 to 500, 5 per line, that are divisible by 3 and 4. Numbers are separated by exactly one space.

المطلوب

البرنامج بيعدّي على كل الأعداد من 1 لحد 500. كل عدد منهم «مرشح» (candidate). المرشح يتقبل لو بيقبل القسمة على 3 وكمان بيقبل القسمة على 4، الشرطين مع بعض. الأعداد المقبولة بس هي اللي تتطبع، خمسة في كل سطر، وبين كل رقمين مسافة واحدة بالظبط: لا مسافتين، ولا مسافة زيادة في آخر السطر.

الخلفية

«العدد يقبل القسمة على 3» معناها إن باقي قسمته على 3 صفر. مثلاً 12 على 3 بـ 4 والباقي صفر، فـ 12 بيقبل القسمة على 3. لكن 10 على 3 بـ 3 والباقي 1، فـ 10 ما بيقبلش.

والعدد اللي بيقبل القسمة على 3 و 4 مع بعض هو بالظبط اللي بيقبل القسمة على 12. ليه 12؟ لأن 3 و 4 ما بينهمش أي عامل مشترك غير الواحد، فأصغر عدد بيقبل القسمة على الاتنين هو حاصل ضربهم، 12 = 4 × 3. ده اسمه المضاعف المشترك الأصغر (LCM). خلي بالك إن القاعدة دي مش دايماً حاصل الضرب: 4 و 6 مثلاً بينهم عامل مشترك 2.

من C++ هنحتاج

العملية `%` اللي بتجيب باقي القسمة، والمقارنة `==`، والربط بين شرطين بـ `&&` (و) أو `||` (أو)، وعداد للأعداد اللي اتطبعت فعلاً علشان نعرف إمتى ننزل سطر.

نبني الحل خطوة خطوة

> كل كود صغير في الجزء ده بيتجرب لوحده: حطوه مكان السطر `// one short example here` جوه `main` في الإطار اللي في أول الصفحة، في برنامج جديد لكل مثال. ولو تحت الكود فيه «الإدخال»، اكتبوه لما البرنامج يستناكم.

الباقي بـ `%`

نطبع لكل عدد من 1 لـ 12: العدد، وباقيه على 3، وباقيه على 4.

```cpp
for (int i = 1; i <= 12; ++i)
    cout << i << "  " << i % 3 << "  " << i % 4 << endl;
```

الخرج

```
1  1  1
2  2  2
3  0  3
4  1  0
5  2  1
6  0  2
7  1  3
8  2  0
9  0  1
10  1  2
11  2  3
12  0  0
```

العمود التاني صفر عند 3 و 6 و 9 و 12، يعني دول بيقبلوا القسمة على 3. العمود التالت صفر عند 4 و 8 و 12. العدد الوحيد اللي العمودين عنده صفر هو `12` .

كل شرط لوحده، وبعدين الاتنين

المقارنة في C++ بتطلّع قيمة صح أو غلط، ولما نطبعها بتظهر `1` للصح و `0` للغلط. نجرب على 8 وعلى 12: الشرط الأول، والتاني، وبعدين `&&` ، وبعدين `||` .

```cpp
int i = 8;
cout << (i % 3 == 0) << " " << (i % 4 == 0) << " "
     << ((i % 3 == 0) && (i % 4 == 0)) << " "
     << ((i % 3 == 0) || (i % 4 == 0)) << endl;
i = 12;
cout << (i % 3 == 0) << " " << (i % 4 == 0) << " "
     << ((i % 3 == 0) && (i % 4 == 0)) << " "
     << ((i % 3 == 0) || (i % 4 == 0)) << endl;
```

الخرج

```
0 1 0 1
1 1 1 1
```

8 مش بتقبل القسمة على 3 (`0`) لكن بتقبل على 4 (`1`). `&&` محتاجة الاتنين صح، فطلعت `0` . أما `||` فيكفيها واحد، فطلعت `1` . و 12 الشرطين عنده صح، فالاتنين طلعوا `1` .

يعني لو كتبنا `||` غلط، هنقبل أعداد مش مطلوبة:

غلط مقصود: `||` بدل `&&`

```
for (int i = 1; i <= 12; ++i)
    if ((i % 3 == 0) || (i % 4 == 0))
        cout << i << " ";
cout << endl;
```

الخرج

```
3 4 6 8 9 12
```

من 1 لـ 12 بس اتقبلت ست أعداد، وخمسة منهم غلط. السؤال بيقول «3 *and* 4»، يعني `&&` :

خطوة في النص: الأرقام صح، لكن الشكل لسه مش المطلوب

```
for (int i = 1; i <= 60; ++i)
    if ((i % 3 == 0) && (i % 4 == 0))
        cout << i << " ";
cout << "|" << endl;
```

الخرج

```
12 24 36 48 60 |
```

دلوقتي الأعداد صح. بس بصوا قبل `|` : فيه مسافة زيادة بعد `60` ، لأن كل رقم بيطبع مسافة بعده، حتى الأخير. والكل في سطر واحد.

نعدّ اللي اتطبع، مش المرشحين

علشان ننزل سطر بعد كل خمس أرقام، محتاجين عداد بيزيد واحد كل ما نطبع رقم: `numbers_per_line` . العداد ده لازم يبقى جوه الـ `if` ، مع الطباعة. وبنستخدمه في حاجتين:

- **قبل الطباعة:** لو العداد مش مضاعف للخمسة (`numbers_per_line % 5 != 0`)، يبقى فيه رقم قبلنا في نفس السطر، فنطبع مسافة الأول. لو مضاعف للخمسة (صفر، أو خمسة، أو عشرة...)، يبقى إحنا أول رقم في السطر، فما فيش مسافة.
- **بعد الطباعة:** نزود العداد، ولو بقى مضاعف للخمسة نبقى طبعنا خمسة في السطر ده، فننزل سطر.

كده المسافة بتتحط بين الأرقام بس، عمرها ما بتيجي في الأول ولا في الآخر. ده جدول بيتابع أول خمس أعداد مقبولة (القيم من التتبع المسجل):

القيمة المقبولة	العداد قبلها	مسافة قبلها؟	العداد بعدها	سطر جديد بعدها؟
12	0	لا (أول الصف)	1	لا
24	1	أيوه	2	لا
36	2	أيوه	3	لا
48	3	أيوه	4	لا
60	4	أيوه	5	أيوه

وبعد ما الحلقة تخلص فيه سطر أخير: `if (numbers_per_line % 5 != 0) cout << endl;` . لو آخر سطر ناقص (فيه أقل من خمسة)، ما حدش نزّل سطر بعده، فبننزله إحنا.

البرنامج كامل

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    int numbers_per_line = 0;

    // 1. visit each candidate in the requested range
    for (int i = 1; i <= 500; ++i)
    {
        // 2. accept only values satisfying both divisibility tests
        if ((i % 3 == 0) && (i % 4 == 0))
        {
            // 3. separate accepted values and count the output row
            if (numbers_per_line % 5 != 0) cout << " ";
            cout << i;
            numbers_per_line++;
            if (numbers_per_line % 5 == 0) cout << endl;
        }
    }
    if (numbers_per_line % 5 != 0) cout << endl;

    return 0;
}
```

الخرج

```
12 24 36 48 60
72 84 96 108 120
132 144 156 168 180
192 204 216 228 240
252 264 276 288 300
312 324 336 348 360
372 384 396 408 420
432 444 456 468 480
492
```

اتقبل 41 عدد: تمن سطور كاملة، وسطر أخير فيه `492` لوحده، لأن مضاعف الـ 12 اللي بعده بيعدّي الـ 500. وبين كل رقمين مسافة واحدة، وما فيش مسافة في أول أي سطر ولا في آخره.

حل تاني: `12 %`

زي ما قلنا في الخلفية، الشرطين مع بعض بيساووا شرط واحد هنا:

```
if (i % 12 == 0)
```

البرنامج البديل طالع نفس الخرج حرف بحرف. الطريقة الأولى أوضح في القراية لأنها بتقول نص السؤال زي ما هو. والتانية أقصر، بس صحتها معتمدة على إن 3 و 4 ما بينهمش عامل مشترك، فلو اتغيرت الأرقام لازم تحسب الـ LCM من جديد.

البرنامج البديل كامل

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    int numbers_per_line = 0;

    // 1. visit each candidate in the requested range
    for (int i = 1; i <= 500; ++i)
    {
        // 2. accept only values satisfying both divisibility tests
        if (i % 12 == 0)
        {
            // 3. separate accepted values and count the output row
            if (numbers_per_line % 5 != 0) cout << " ";
            cout << i;
            numbers_per_line++;
            if (numbers_per_line % 5 == 0) cout << endl;
        }
    }
    if (numbers_per_line % 5 != 0) cout << endl;

    return 0;
}
```

ممكن تقع في ده

- **تكتب `||` .** هتقبل كل مضاعفات الـ 3 وكل مضاعفات الـ 4، زي ما شفنا مع 1 لـ 12.
- **تعدّ المرشحين بدل المطبوع.** لو العداد بيزيد مع كل `i` مش مع كل رقم اتطبع، السطور هتتقطع على حسب المرشحين:

غلط مقصود: العداد بيعدّ المرشحين

```
int count = 0;
for (int i = 1; i <= 30; ++i)
{
    if ((i % 3 == 0) && (i % 4 == 0)) cout << i << " ";
    count++;
    if (count % 5 == 0) cout << endl;
}
```

الخرج

```


12

24
```

من 1 لـ 30 اتقبل رقمين بس، لكن العداد نزّل سطر كل خمس مرشحين، فطلعت سطور فاضية.

- **تطبع مسافة بعد كل رقم.** آخر رقم في كل سطر هيبقى بعده مسافة مش مطلوبة. السؤال قال «exactly one space» بين الأرقام.
- **تجمع `setw` مع مسافة.** `setw(3)` بتكمّل الأرقام الصغيرة بمسافات، ولو زودت `" "` فوقها هيبقى فيه مسافتين بين الأرقام اللي من رقمين. ده اللي حصل في حل المصدر (تحت).
- **تنسى السطر الأخير.** من غير `if` اللي بعد الحلقة، الخرج هيخلص من غير سطر جديد بعد `492` .

ملاحظة على حل المصدر

حل المصدر بيختار نفس الأعداد بالظبط (بشرط `i % 12 == 0`)، لكنه بيطبع كل رقم في `setw(3)` وبعده مسافة. ده أول سطر من طريقته، و `|` بتعلم آخر السطر:

للمقارنة: طريقة المسافات في حل المصدر

```
int numbers_per_line = 0;
for (int i = 1; i <= 60; ++i)
    if (i % 12 == 0)
    {
        cout << setw(3) << i;
        numbers_per_line++;
        if (numbers_per_line % 5 == 0) cout << "|" << endl;
        else cout << " ";
    }
```

الخرج

```
 12  24  36  48  60|
```

بصوا على السطر ده بالظبط: قبل `12` مسافة واحدة، جاية من `setw(3)` لأن 12 رقمين والخانة تلاتة. وبعد كده بين كل رقمين ورا بعض مسافتين: واحدة من `" "` اللي اتطبعت بعد الرقم اللي قبله، وواحدة من `setw(3)` للرقم نفسه. وفي حل المصدر كمان، آخر سطر الناقص بيخلص بمسافة زيادة بعد `492` ، لأن المسافة بتتطبع بعد أي رقم مش خامس في سطره. والشيت طالب مسافة واحدة بالظبط بين الأرقام، فالبرنامج هنا شال `setw` وبيحط المسافة بين الأرقام بس.

---

9 من 300 لـ 400: على 3 ومش على 6

(9) Write a C++ program that displays all the numbers from 300 to 400, 5 per line, that are divisible by 3 but not divisible by 6. Numbers are separated by exactly one space.

المطلوب

نعدّي على الأعداد من 300 لحد 400، الاتنين داخلين. العدد يتقبل لو بيقبل القسمة على 3، وفي نفس الوقت *ما* بيقبلش القسمة على 6. المقبول بس يتطبع، خمسة في كل سطر، وبين كل رقمين مسافة واحدة بالظبط.

الخلفية

أي مضاعف لـ 6 هو مضاعف لـ 3 كمان (2 × 3 = 6). يعني مضاعفات الـ 3 نوعين: الزوجي منها بيقبل القسمة على 6، والفردي منها ما بيقبلش. فالمطلوب فعلياً هو مضاعفات الـ 3 الفردية. خدوا تلات أعداد ورا بعض من مضاعفات الـ 3: 300 زوجي فبيقبل على 6 ومش مطلوب، 303 فردي ومطلوب، 306 زوجي ومش مطلوب.

من C++ هنحتاج

`%` و`==` زي السؤال 8، وكمان `!=` (لا يساوي) علشان كلمة «not»، والربط بـ `&&`، ونفس طريقة طباعة خمسة في السطر بعداد للمطبوع.

نبني الحل خطوة خطوة

> كل كود صغير في الجزء ده بيتجرب لوحده: حطوه مكان السطر `// one short example here` جوه `main` في الإطار اللي في أول الصفحة، في برنامج جديد لكل مثال. ولو تحت الكود فيه «الإدخال»، اكتبوه لما البرنامج يستناكم.

تلات مرشحين

```
for (int i = 300; i <= 306; i = i + 3)
    cout << i << "  " << i % 3 << "  " << i % 6 << endl;
```

الخرج

```
300  0  0
303  0  3
306  0  0
```

التلاتة باقيهم على 3 صفر. الفرق في باقيهم على 6: 300 و 306 باقيهم صفر، و 303 باقيه 3. يعني الشرط التاني هو اللي هيفرق.

نغير مقارنة واحدة: `==` تبقى `!=`

نكتب الشرطين بطريقتين، وكل حاجة ثابتة ما عدا المقارنة التانية. العمود الأول بـ `i % 6 == 0`، والتاني بـ `i % 6 != 0`:

العمود الأول (`== 0`) عكس المطلوب، والتاني (`!= 0`) هو الصح

```
for (int i = 300; i <= 306; i = i + 3)
{
    cout << i << ": ";
    cout << ((i % 3 == 0) && (i % 6 == 0)) << " ";
    cout << ((i % 3 == 0) && (i % 6 != 0)) << endl;
}
```

الخرج

```
300: 1 0
303: 0 1
306: 1 0
```

العمود الأول اختار 300 و 306، وده عكس المطلوب. لما غيرنا `==` لـ `!=` بس، الاختيار اتقلب واتقبل 303 لوحده. كلمة «not divisible by 6» معناها إن الباقي على 6 مش صفر، وده بالظبط `i % 6 != 0`.

وممكن نكتب «مش» بطريقة تانية: علامة `!` قبل الشرط كله بتقلبه. الطريقتين نفس النتيجة:

```
int i = 303;
cout << (i % 6 != 0) << " " << !(i % 6 == 0) << endl;
i = 306;
cout << (i % 6 != 0) << " " << !(i % 6 == 0) << endl;
```

الخرج

```
1 1
0 0
```

الطباعة خمسة في السطر

نفس فكرة السؤال 8، ونعيدها هنا باختصار علشان السؤال يتقري لوحده. عندنا عداد `numbers_per_line` بيزيد بس لما نطبع رقم مقبول:

- قبل ما نطبع: لو العداد مش مضاعف للخمسة، يبقى فيه رقم قبلنا في السطر، فنطبع مسافة.
- نطبع الرقم ونزود العداد.
- لو العداد بقى مضاعف للخمسة، نبقى كملنا خمسة، فننزل سطر.
- بعد الحلقة: لو آخر سطر ناقص، ننزل سطر.

البرنامج كامل

```
#include<iostream>
using namespace std;

int main() {
    int numbers_per_line = 0;

    // 1. visit each candidate in the requested range
    for (int i = 300; i <= 400; ++i)
    {
        if ((i % 3 == 0) && (i % 6 != 0))
        {
            // 2. separate only the values actually printed
            if (numbers_per_line % 5 != 0) cout << " ";
            cout << i;
            numbers_per_line++;
            if (numbers_per_line % 5 == 0) cout << endl;
        }
    }
    if (numbers_per_line % 5 != 0) cout << endl;
    return 0;
}
```

الخرج

```
303 309 315 321 327
333 339 345 351 357
363 369 375 381 387
393 399
```

اتقبل 17 عدد: تلات سطور كاملة، وسطر أخير فيه رقمين بس (`393 399`). مفيش ولا رقم زوجي، ومفيش مسافة بعد `399` .

حل تاني: الباقي على 6 يساوي 3

مضاعف الـ 3 اللي مش مضاعف لـ 6 باقيه على 6 لازم يبقى 3 (الباقي على 6 ممكن يبقى 0 لحد 5، ومضاعف الـ 3 باقيه 0 أو 3 بس). فالشرطين ممكن يبقوا شرط واحد:

```
if (i % 6 == 3)
```

البديل طالع نفس الخرج بالظبط. لكن الطريقة دي مضمونة هنا لأن كل الأعداد من 300 لـ 400 موجبة. مع الأعداد السالبة، الباقي في C++ ممكن يطلع سالب:

تجربة بعدد سالب، بره مدى السؤال

```
int i = -3;
cout << i % 6 << endl;
cout << (i % 6 == 3) << " " << ((i % 3 == 0) && (i % 6 != 0)) << endl;
```

الخرج

```
-3
0 1
```

3- مضاعف لـ 3 ومش مضاعف لـ 6، يعني المفروض يتقبل. الشرطين الأصليين قبلوه (`1`)، لكن `3 == 6 %` رفضته (`0`) لأن الباقي طلع `-3` مش 3. فالشرط المختصر صح في حدود السؤال ده بس.

البرنامج البديل كامل

```cpp
#include<iostream>
using namespace std;

int main() {
    int numbers_per_line = 0;

    // 1. visit each candidate in the requested range
    for (int i = 300; i <= 400; ++i)
    {
        if (i % 6 == 3)
        {
            // 2. separate only the values actually printed
            if (numbers_per_line % 5 != 0) cout << " ";
            cout << i;
            numbers_per_line++;
            if (numbers_per_line % 5 == 0) cout << endl;
        }
    }
    if (numbers_per_line % 5 != 0) cout << endl;
    return 0;
}
```

ممكن تقع في ده

- **تكتب `i % 6 == 0`.** ده «بيقبل القسمة على 6»، عكس المطلوب.
- **تنزل سطر على حسب قيمة الرقم.** لو كتبت `if (i % 5 == 0) cout << endl;`، السطر هيتقطع عند الأرقام اللي بتقبل القسمة على 5، مش بعد كل خمس أرقام:

غلط مقصود: السطر بيتقطع على حسب قيمة الرقم

```cpp
for (int i = 300; i <= 400; ++i)
    if ((i % 3 == 0) && (i % 6 != 0))
    {
        cout << i << " ";
        if (i % 5 == 0) cout << endl;
    }
cout << endl;
```

الخرج

```
303 309 315
321 327 333 339 345
351 357 363 369 375
381 387 393 399
```

أول سطر فيه تلاتة بس، وبعد كل سطر مسافة زيادة. العداد لازم يعدّ المطبوع.

- **تكتب `||` بدل `&&`.** «مضاعف لـ 3 *أو* مش مضاعف لـ 6» بيقبل كل الأعداد من غير استثناء. أي عدد يا إما مضاعف لـ 6، وساعتها هو مضاعف لـ 3 فبيعدّي من الشرط الأول، يا إما مش مضاعف لـ 6، فبيعدّي من الشرط التاني:

غلط مقصود: `||` بدل `&&`

```
for (int i = 300; i <= 306; ++i)
    if ((i % 3 == 0) || (i % 6 != 0))
        cout << i << " ";
cout << endl;
```

الخرج

```
300 301 302 303 304 305 306
```

كل الأعداد من 300 لـ 306 اتقبلت.

- **تبدأ من 301 أو تقف عند 399.** «from 300 to 400» معناها الاتنين داخلين في الفحص، حتى لو 300 نفسه مش هيتقبل.
- **تسيب مسافة بعد آخر رقم.** آخر سطر هنا ناقص (رقمين)، ولو كل رقم بيطبع مسافة بعده، هيبقى فيه مسافة زيادة بعد `399` .

ملاحظة على حل المصدر

حل المصدر بنفس الشرط وبيطلّع نفس الأعداد. الفرق الوحيد إنه بيطبع المسافة بعد الرقم، فآخر سطر الناقص بيخلص بمسافة بعد `399` . البرنامج هنا بيحط المسافة قبل الرقم، فما بتظهرش في الآخر.

---

10 من كود ASCII للحرف

(10) Write a C++ program that receives an ASCII code (an integer between 0 and 127) and displays its character.
A sample run gives the following output;

```
Enter an ASCII code (0-127): 69
The character for ASCII code 69 is: E
```

المطلوب

البرنامج يطلب من المستخدم رقم صحيح بين 0 و 127، وده كود ASCII، ويطبع الحرف اللي الكود ده بيمثله. المثال في الشيت: المستخدم كتب `69` ، والبرنامج طبع `E` . والسؤال حدد المدى، فأي رقم بره 0 لـ 127 مش كود ASCII.

الخلفية

الكمبيوتر ما بيخزنش حروف، بيخزن أرقام بس. علشان يبقى فيه حروف، لازم يبقى فيه اتفاق: الرقم ده معناه الحرف ده. الاتفاق ده اسمه ترميز (encoding)، وASCII من أقدم الاتفاقات وأشهرها. فيه 128 كود، من 0 لـ 127. شوية أمثلة من الاتفاق:

- الحروف الكبيرة `A` لـ `Z` : الأكواد 65 لـ 90، ورا بعض.
- الحروف الصغيرة `a` لـ `z` : الأكواد 97 لـ 122.
- الأرقام كحروف `'0'` لـ `'9'` : الأكواد 48 لـ 57.
- الأكواد من 0 لـ 31، والكود 127: دي «حروف تحكم» (control characters)، زي سطر جديد. معظمها ما لهاش شكل يترسم على الشاشة. الكود 0 اسمه NUL والكود 127 اسمه DEL.

من C++ هنحتاج

متغير `int` للكود، ومتغير `char` للحرف، والتحويل بينهم بـ `static_cast<char>`، و`if` مع `||` علشان نرفض أي رقم بره المدى.

نبني الحل خطوة خطوة

> كل كود صغير في الجزء ده بيتجرب لوحده: حطوه مكان السطر `// one short example here` جوه `main` في الإطار اللي في أول الصفحة، في برنامج جديد لكل مثال. ولو تحت الكود فيه «الإدخال»، اكتبوه لما البرنامج يستناكم.

نفس الرقم، طريقتين عرض

```
int code = 69;
cout << code << endl;
cout << static_cast<char>(code) << endl;
```

الخرج

```
69
E
```

القيمة هي هي في السطرين: 69. الفرق في النوع. `cout` لما بياخد `int` بيطبع الرقم، ولما بياخد `char` بيطبع الحرف اللي الرقم بيمثله. و`static_cast<char>(code)` معناها «اتعامل مع القيمة دي على إنها `char`». الاتجاه هنا من رقم لحرف، والسؤال 11 هيعمل العكس.

جيران الـ 69

```
for (int code = 65; code <= 70; code++)
    cout << code << " " << static_cast<char>(code) << endl;
cout << 48 << " " << static_cast<char>(48) << endl;
cout << 97 << " " << static_cast<char>(97) << endl;
```

الخرج

```
65 A
66 B
67 C
68 D
69 E
70 F
48 0
97 a
```

الحروف الكبيرة ورا بعض: 65 هو A، و 66 هو B، وهكذا. وآخر سطرين بيبينوا حاجتين: الكود 48 هو *الحرف* `0` (مش الرقم صفر)، والكود 97 هو `a` الصغيرة.

نرفض اللي بره المدى

البرنامج بيسأل قبل التحويل:

```
if (ASCII_Code < 0 || ASCII_Code > 127)
{
    cout << "Invalid ASCII code." << endl;
    return 0;
}
```

الرقم بيبقى بره المدى لو أصغر من 0 *أو* أكبر من 127، فالربط بـ `||` . ونجرب أول رقم بره من كل ناحية:

الإدخال

```
-1
```

الخرج

```
Enter an ASCII code (0-127): Invalid ASCII code.
```

الإدخال

```
128
```

الخرج

```
Enter an ASCII code (0-127): Invalid ASCII code.
```

أطراف المدى: 0 و 127

الرقمين دول جوه المدى، فالبرنامج بيقبلهم ويطبع الحرف. بس الحرف حرف تحكم، مالوش شكل:

الإدخال

```
0
```

الخرج

```
Enter an ASCII code (0–127): The character for ASCII code 0 is: NUL
```

الإدخال

```
127
```

الخرج

```
Enter an ASCII code (0–127): The character for ASCII code 127 is: DEL
```

البرنامج فعلاً طبع بايت قيمته 0 في الأول، وبايت قيمته 127 في التاني. إحنا كتبنا مكانهم NUL و DEL في الصفحة علشان تشوفوهم، لكن في نافذة التشغيل العادية الرسالة نفسها هتظهر كاملة، ومكان الحرف ده بس مش هيبقى ليه شكل مرئي عادي. التحويل ما فشلش؛ ده الحرف المتفق عليه للكود ده، وهو حرف تحكم مالوش رسمة.

البرنامج كامل

```cpp
#include <iostream>
using namespace std;

int main() {
    int ASCII_Code;
    char character;

    // 1. read an ASCII code
    cout << "Enter an ASCII code (0–127): ";
    if (!(cin >> ASCII_Code)) return 0;
    // 2. enforce the input range before converting
    if (ASCII_Code < 0 || ASCII_Code > 127)
    {
        cout << "Invalid ASCII code." << endl;
        return 0;
    }
    character = static_cast<char>(ASCII_Code);
    cout << "The character for ASCII code "
         << ASCII_Code << " is: " << character << endl;

    return 0;
}
```

الإدخال

```
69
```

الخرج

```
Enter an ASCII code (0–127): The character for ASCII code 69 is: E
```

ده نفس مثال الشيت. الفرق إن الـ 69 اللي اتكتبت مش ظاهرة بعد الرسالة هنا، لأن الإدخال اتبعت جاهز زي ما قلنا في أول الصفحة. لما تشغلوا البرنامج وتكتبوا من الكيبورد في نافذة تشغيل عادية، اللي بتكتبوه بيظهر عادةً بعد الرسالة، زي صورة الشيت.

حل تاني: `char(code)`

C++ فيها طريقة تانية أقصر لنفس التحويل، شكلها زي استدعاء دالة:

```
character = char(ASCII_Code);
```

في الحالة دي `char(ASCII_Code)` و `static_cast<char>(ASCII_Code)` نفس التحويل بالظبط. جربنا البديل على الخمس إدخالات (69، 0، 127، 1–، 128) وطلع نفس الخرج بايت ببايت. الفرق إن `static_cast` مكتوبة بوضوح إنها تحويل، فأسهل تلاقيها وانت بتقرا الكود.

البرنامج البديل كامل

```
#include <iostream>
using namespace std;

int main() {
    int ASCII_Code;
    char character;

    // 1. read an ASCII code
    cout << "Enter an ASCII code (0–127): ";
    if (!(cin >> ASCII_Code)) return 0;
    // 2. enforce the input range before converting
    if (ASCII_Code < 0 || ASCII_Code > 127)
    {
        cout << "Invalid ASCII code." << endl;
        return 0;
    }
    character = char(ASCII_Code);
    cout << "The character for ASCII code "
         << ASCII_Code << " is: " << character << endl;

    return 0;
}
```

ممكن تقع في ده

- **تقرا الكود في `char`.** لو المتغير اللي بتقرا فيه `char` ، `cin` هياخد حرف واحد بس من اللي اتكتب:

غلط مقصود: الكود اتقرا في `char`

```
char code;
cin >> code;
cout << code << endl;
```

الإدخال

```
69
```

الخرج

```
6
```

اتكتب 69، واتقرا الحرف `6` بس. الكود رقم، يتقري في `int` .

- **تكتب الشرط بـ `&&`.** `code < 0 && code > 127` عمره ما هيبقى صح، مفيش رقم أصغر من صفر وأكبر من 127 في نفس الوقت، فمش هترفض حاجة.
- **تطبع المتغير الغلط.** لو طبعت `ASCII_Code` مكان `character` ، هيطلع 69 تاني بدل E.
- **تفتكر إن غياب الحرف معناه غلطة.** الأكواد 0 لـ 31 و 127 حروف تحكم. الرسالة بتتطبع عادي، والحرف نفسه بس مالوش شكل مرئي؛ ده نتيجة صحيحة.
- **تحط `-1` أو `200` وتستنى حرف.** دول بره ASCII. السؤال حدد 0 لـ 127.

ملاحظة على حل المصدر

حل المصدر بيعمل التحويل على طول من غير ما يتأكد إن الرقم في المدى. السؤال بيقول «an integer between 0 and 127»، فالبرنامج هنا زوّد السؤال ده قبل التحويل. وزود كمان سطر بيقفل البرنامج لو القراية فشلت (اتكتب حرف بدل رقم مثلاً).

11 من الحرف لكود ASCII

(11) Write a program that receives a character and displays its ASCII code.
A sample run gives the following output;

```
Enter a character: D
ASCII Value of D is 68
```

المطلوب

عكس السؤال 10. البرنامج يقرا حرف واحد، ويطبع كود ASCII بتاعه. المثال في الشيت: اتكتب `D` ، والبرنامج طبع `68` . المقصود هنا حرف ASCII عادي (حرف إنجليزي، أو رقم، أو علامة) من غير مسافات.

الخلفية

زي ما قلنا في السؤال 10، كل حرف متخزن كرقم حسب جدول ASCII. السؤال ده بيطلب نبص على الرقم المتخزن بدل الحرف. وفيه نقطة بتلخبط ناس كتير: الحرف `'7'` (اللي بتكتبه على الكيبورد) مش هو الرقم 7. الحرف `'7'` ليه كود في الجدول، والأرقام كحروف ورا بعض في الجدول من `'0'` لـ `'9'` ، وده اللي هيخلينا نجيب قيمة الرقم بطرح بسيط.

من C++ هنحتاج

متغير `char` بنقرا فيه بـ `cin >>` ، والتحويل لـ `int` بـ `int(c)` أو `static_cast<int>(c)` ، والفرق بين علامة التنصيص المفردة `'D'` (حرف واحد، `char`) والمزدوجة `"D"` (نص).

نبني الحل خطوة خطوة

> كل كود صغير في الجزء ده بيتجرب لوحده: حطوه مكان السطر `// one short example here` جوه `main` في الإطار اللي في أول الصفحة، في برنامج جديد لكل مثال. ولو تحت الكود فيه «الإدخال»، اكتبوه لما البرنامج يستناكم.

نفس القيمة بالعكس

```
char c = 'D';
cout << c << endl;
cout << int(c) << endl;
```

الخرج

```
D
68
```

المتغير `c` نوعه `char`، فـ `cout` طبعه حرف: `D`. ولما قلنا `int(c)`، طبع نفس القيمة كرقم: `68`. ما حصلش حساب؛ إحنا بس غيرنا طريقة العرض. و `'D'` بعلامة مفردة معناها حرف واحد قيمته 68. لو كتبتها `"D"` بعلامتين، تبقى نص (string)، وده نوع تاني خالص.

الحرف `'7'` والرقم 7

```
char c = '7';
cout << c << " " << int(c) << " " << c - '0' << endl;
```

الخرج

```
7 55 7
```

الحرف `7` كوده `55`. ولو طرحنا منه كود الحرف `'0'` (اللي هو 48)، يطلع `7`، قيمة الرقم الحقيقية. ده بيشتغل لأن الأرقام كحروف ورا بعض في الجدول: `'0'` هو 48 و `'9'` هو 57 والباقي بينهم بالترتيب، فالمسافة بين أي حرف رقم و `'0'` هي قيمته. السؤال ده عايز الكود (`55`)، مش القيمة.

`cin >>` مع `char` بيقرا إيه بالظبط

البرنامج الكامل بيقرا بـ `cin >> c`. القراية دي بتعدّي أي مسافات أو أسطر فاضية في الأول، وتاخد أول حرف بعدها، حرف واحد بس. نجرب البرنامج الكامل بإدخال قبله تلات مسافات (العلامة `␣` معناها مسافة):

الإدخال

```
␣␣␣D
```

الخرج

```
Enter a character: ASCII Value of D is 68
```

المسافات اتعدّت، واتقرا `D`. ولو اتكتب حرفين:

الإدخال

```
DE
```

الخرج

```
Enter a character: ASCII Value of D is 68
```

اتقرا أول حرف بس، والـ `E` فضلت مستنية في الإدخال من غير ما حد يقراها. ولو اتكتبت مسافة لوحدها:

الإدخال

```
␣
```

الخرج

```
Enter a character:
```

في التشغيل المسجل ده، الإدخال كان ملف جاهز وخلص بعد المسافة. `cin >>` عدّى المسافة ودوّر على حرف، ولقى الإدخال انتهى، فالقراية فشلت والبرنامج قفل من غير ما يطبع النتيجة. لكن لما تشغلوا البرنامج وتكتبوا من الكيبورد، الإدخال بيفضل مفتوح، فـ `cin >>` مش بيلاقي نهاية: بيعدّي المسافة (وحتى Enter) ويفضل مستني لحد ما يتكتب حرف مش مسافة. في الحالتين، المسافة نفسها ما بتتقريش بالطريقة دي.

البرنامج كامل

```cpp
#include <iostream>
using namespace std;

int main() {
    char c;
    cout << "Enter a character: ";
    if (!(cin >> c)) return 0;
    // 2. print the numeric character code
    cout << "ASCII Value of " << c << " is " << int(c) << endl;
    return 0;
}
```

الإدخال

```
D
```

الخرج

```
Enter a character: ASCII Value of D is 68
```

ونجرب أول وآخر حرف كبير، علشان نتأكد من المدى اللي هنستخدمه في السؤال 14:

الإدخال

```
A
```

الخرج

```
Enter a character: ASCII Value of A is 65
```

الإدخال

```
Z
```

الخرج

```
Enter a character: ASCII Value of Z is 90
```

والحرف `7` :

الإدخال

```
7
```

الخرج

```
Enter a character: ASCII Value of 7 is 55
```

حل تاني: `static_cast<int>`

```
cout << "ASCII Value of " << c << " is " << static_cast<int>(c) << endl;
```

`static_cast<int>(c)` و `int(c)` نفس التحويل هنا، والبديل طلع نفس الخرج في كل الإدخالات اللي جربناها، حتى المسافات والحرفين. الفرق في القراية بس: `static_cast` بتقول صراحة «ده تحويل نوع».

البرنامج البديل كامل

```
#include <iostream>
using namespace std;

int main() {
    char c;
    cout << "Enter a character: ";
    if (!(cin >> c)) return 0;
    // 2. print the numeric character code
    cout << "ASCII Value of " << c << " is " << static_cast<int>(c) << endl;
    return 0;
}
```

ممكن تقع في ده

- **تطبع `c` من غير تحويل.** هيطبع الحرف نفسه تاني، مش الكود.
- **تستنى إن `'7'` يطبع 7.** كوده 55. القيمة 7 بتيجي من `c - '0'` ، والسؤال ده عايز الكود.
- **تكتب `"D"` بعلامتين.** ده نص مش حرف، وما ينفعش تحطه في `char` .
- **تستنى إن المسافة تتقري.** `cin >>` بيعدّي المسافات. لو عايز تقرا مسافة محتاج طريقة قراية تانية، وده بره السؤال ده.
- **تكتب حرف عربي.** الحروف العربية مش في ASCII، وغالباً بتتخزن في أكتر من بايت، فالنتيجة مش هتبقى كود ASCII. السؤال بيتكلم عن ASCII بس.

ملاحظة على حل المصدر

حل المصدر بيطبع الكود من غير `endl` في الآخر، فالسطر بيفضل مفتوح. البرنامج هنا زود `endl` ، وزود سطر بيقفل البرنامج لو القراية فشلت.

---

12 من حرف كبير لحرف صغير

(12) Write a C++ program that prompts the user to enter an uppercase letter and converts it to a lowercase letter.

المطلوب

البرنامج يطلب من المستخدم حرف إنجليزي كبير (uppercase)، ويطبع نفس الحرف صغير (lowercase): `A` تبقى `a` ، و `Z` تبقى `z` . السؤال طالب «uppercase letter»، فلو اتكتب حاجة تانية (حرف صغير، رقم، علامة)، البرنامج يقول إن الإدخال مش مقبول بدل ما يطبع حاجة ملهاش معنى.

الخلفية

في جدول ASCII الحروف الكبيرة من 65 لـ 90، والصغيرة من 97 لـ 122، والاتنين بنفس الترتيب. يعني كل حرف صغير بعيد عن الكبير بتاعه بنفس المسافة بالظبط. المسافة دي هنحسبها في الحل التاني.

والسؤال فيه خطوتين مختلفتين، ومهم ما نخلطهمش: الأولى *سؤال* «هل الحرف ده كبير؟»، والتانية تغيير «حوّله لصغير».

من C++ هنحتاج

المكتبة <cctype> وفيها دالتين: isupper بتسأل، و tolower بتغير. والاتنين بيرجعوا int ، فلازم نفهم إيه اللي بيحصل لما نحط النتيجة في char . ومعاهم if / else .

نبني الحل خطوة خطوة

> كل كود صغير في الجزء ده بيتجرب لوحده: حطوه مكان السطر // one short example here جوه main في الإطار اللي في أول الصفحة، في برنامج جديد لكل مثال. ولو تحت الكود فيه «الإدخال»، اكتبوه لما البرنامج يستناكم.

السؤال: isupper

```
char c = 'A';
cout << isupper(c) << " " << (isupper(c) != 0) << endl;
c = 'a';
cout << isupper(c) << " " << (isupper(c) != 0) << endl;
```

الخرج

```
1 1
0 0
```

لكل حرف طبعنا حاجتين: اللي isupper رجعته على طول، وبعدين المقارنة 0 != . لـ A رجعت 1 على المترجم ده، ولـ a رجعت 0 . المهم تعرفوه: المعيار ما بيوعدش إن الإجابة «أيوه» تبقى 1 بالظبط، بيوعد بس إنها مش صفر. على مترجم تاني ممكن تطلع رقم تاني. علشان كده بنستخدمها جوه if على طول، و if بتعتبر أي رقم مش صفر «صح».

التغيير: tolower

السطر الأول بيطبع الكود (رقم) مش الحرف؛ التاني هو الشكل المطلوب

```
char c = 'A';
cout << tolower(c) << endl;
char lower = tolower(c);
cout << lower << endl;
```

الخرج

```
97
a
```

أول سطر طبع 97 مش a ! لأن tolower بترجع int ، و cout بيطبع الـ int كرقم. والرقم ده هو كود a . في السطر التاني حطينا نفس النتيجة في متغير char ، فاتطبعت حرف: a . علشان كده البرنامج بيكتب char lowercaseChar = tolower(inputChar); .

ليه السؤال لازم ييجي قبل التغيير

`tolower` لوحدها مش بترفض حاجة. لو اديتها حاجة مش حرف كبير، بترجعها زي ما هي:

```
char a = tolower('?');
char b = tolower('a');
cout << a << " " << b << endl;
```

الخرج

```
? a
```

العلامة `?` رجعت `?`، والـ `a` رجعت `a`. يعني لو شلنا `isupper`، البرنامج هيقول «The lowercase letter is: ?»، وده كلام مالوش معنى. اللي بيرفض الإدخال الغلط هو `isupper`.

البرنامج كامل

```
#include <iostream>
#include <cctype> // Required for character handling functions like tolower() and isupper()
using namespace std;

int main() {
    char inputChar;

    // 1. read an uppercase letter
    cout << "Enter an uppercase letter: ";
    if (!(cin >> inputChar)) return 0;

    // 2. check whether the entered character is an uppercase letter
    if (isupper(inputChar)) {
        char lowercaseChar = tolower(inputChar);
        cout << "The lowercase letter is: " << lowercaseChar << endl;
    } else {
        // Inform the user if the input is not a valid uppercase letter
        cout << "Invalid input. Please enter an uppercase letter." << endl;
    }

    return 0;
}
```

أول وآخر حرف كبير:

الإدخال

```
A
```

الخرج

```
Enter an uppercase letter: The lowercase letter is: a
```

الإدخال

```
Z
```

الخرج

```
Enter an uppercase letter: The lowercase letter is: z
```

نغير الإدخال بس لحرف صغير، وبعدين لعلامة. الاتنين اترفضوا من `isupper`:

الإدخال

```
a
```

الخرج

```
Enter an uppercase letter: Invalid input. Please enter an uppercase letter.
```

الإدخال

```
?
```

الخرج

```
Enter an uppercase letter: Invalid input. Please enter an uppercase letter.
```

حل تاني: المسافة الثابتة في ASCII

بما إن كل حرف صغير بعيد عن الكبير بنفس المسافة، نحسبها مرة:

```
cout << ('a' - 'A') << endl;
char c = 'Q';
cout << char(c + ('a' - 'A')) << endl;
```

الخرج

```
32
q
```

المسافة بين `'a'` و`'A'` هي `32`، فلو زودناها على `Q` طلعت `q`. البرنامج البديل بيستخدم الفكرة دي، وبيسأل بمقارنة المدى بدل `isupper`:

```
if (inputChar >= 'A' && inputChar <= 'Z') {
    char lowercaseChar = char(inputChar + ('a' - 'A'));
    cout << "The lowercase letter is: " << lowercaseChar << endl;
} else {
    // Inform the user if the input is not a valid uppercase letter
    cout << "Invalid input. Please enter an uppercase letter." << endl;
}
```

الشرط `inputChar >= 'A' && inputChar <= 'Z'` معناه «الكود بين كود A وكود Z»، والإضافة `('a' - 'A')` بتنقل الحرف للصغير. البديل طلع نفس الخرج في الأربع إدخالات. لكن خدوا بالكم من حدوده: الطريقة دي معتمدة على ترتيب جدول ASCII بالظبط، وبتشتغل مع الحروف الإنجليزية بس. `isupper` و`tolower` مش محتاجين تعرفوا الجدول خالص.

البرنامج البديل كامل

```
#include <iostream>
using namespace std;

int main() {
    char inputChar;

    // 1. read an uppercase letter
    cout << "Enter an uppercase letter: ";
    if (!(cin >> inputChar)) return 0;

    // 2. check whether the entered character is an uppercase letter
    if (inputChar >= 'A' && inputChar <= 'Z') {
        char lowercaseChar = char(inputChar + ('a' - 'A'));
        cout << "The lowercase letter is: " << lowercaseChar << endl;
    } else {
        // Inform the user if the input is not a valid uppercase letter
        cout << "Invalid input. Please enter an uppercase letter." << endl;
    }

    return 0;
}
```

ممكن تقع في ده

- **تطبع `tolower(c)` على طول.** هتطلع `97` مش `a`، زي ما شفنا. حطها في `char` الأول، أو حوّلها: `char(tolower(c))`.
- **تشيل `isupper`.** `tolower` مش هترفض `?` ولا `a`، والبرنامج هيطبع نتيجة ملهاش معنى.
- **تفترض إن `isupper` بترجع `1`.** بترجع «مش صفر». ما تكتبش `if (isupper(c) == 1)`؛ اكتب `if (isupper(c))`.
- **تطرح 32 بدل ما تزودها.** الصغير كوده أكبر من الكبير، فالتحويل لصغير بالإضافة.
- **تديّها حرف مش ASCII.** دوال `<cctype>` معمولة لقيم `char` العادية؛ الحروف العربية مثلاً بره حدود السؤال ده، وممكن تديها قيم سالبة ما ينفعش تتبعت للدوال دي.

حل المصدر بيستخدم نفس `isupper` و`tolower` ونفس الرسايل. البرنامج هنا زود بس سطر بيقفل لو القراية فشلت.

---

13 تصنيف حرف واحد: كبير ولا صغير ولا رقم

(13) Design a C++ program that prompts the user to input a single character, which can be either:

- A letter (uppercase or lowercase), or
- A single-digit number (0-9).

The program should then read the input and display one of the following messages to the user, based on the character type:

- "Uppercase Letter" if the character is an uppercase letter (A-Z).
- "Lowercase Letter" if the character is a lowercase letter (a-z).
- "Number" if the character is a single-digit number (0-9).
- "Invalid Input" if the input is not a letter or a single-digit number.

المطلوب

البرنامج يقرا حرف واحد، ويطبع رسالة واحدة بس من أربعة على حسب نوعه:

- `Uppercase Letter` لو حرف كبير من A لـ Z.
- `Lowercase Letter` لو حرف صغير من a لـ z.
- `Number` لو رقم واحد من 0 لـ 9.
- `Invalid Input` لو أي حاجة تانية.

الكلمة المهمة: «single character». البرنامج بيقرا حرف واحد، مش كلمة ولا رقم من كذا خانة.

الخلفية

كل نوع من التلاتة مدى متصل في جدول ASCII: الكبير من 65 لـ 90، والصغير من 97 لـ 122، والأرقام كحروف من 48 لـ 57. وبين المديات دي فيه علامات. الجيران اللي على حدود كل مدى بالظبط مش من أي نوع:

قبل المدى	المدى	بعد المدى
`/` (47)	`'0'` لـ `'9'` (48 لـ 57)	`:` (58)
`@` (64)	`A` لـ `Z` (65 لـ 90)	`[` (91)
`` ` `` (96)	`a` لـ `z` (97 لـ 122)	`{` (123)

الجيران دول بالظبط هما اللي هنجرب بيهم الحل التاني، لأن أي غلطة في حدود المدى هتبان عندهم.

من C++ هنحتاج

سلسلة `if / else if / else` ، ودوال `<cctype>` : `isupper` و `islower` و `isdigit` . كل دالة منهم بترجع رقم مش صفر لو الإجابة أيوه، وصفر لو لأ، زي ما شفنا مع `isupper` في السؤال 12.

نبني الحل خطوة خطوة

> كل كود صغير في الجزء ده بيتجرب لوحده: حطوه مكان السطر `// one short example here` جوه `main` في الإطار اللي في أول الصفحة، في برنامج جديد لكل مثال. ولو تحت الكود فيه «الإدخال»، اكتبوه لما البرنامج يستناكم.

نمشي حرف واحد في الأسئلة التلاتة

ناخد `A` ونسأله التلات أسئلة، وبعدين `?` . طبعنا كل إجابة كـ 1 أو 0 علشان تبان:

```
char c = 'A';
cout << c << ": " << (isupper(c) != 0) << " "
     << (islower(c) != 0) << " " << (isdigit(c) != 0) << endl;
c = '?';
cout << c << ": " << (isupper(c) != 0) << " "
     << (islower(c) != 0) << " " << (isdigit(c) != 0) << endl;
```

الخرج

```
A: 1 0 0
?: 0 0 0
```

`A` : أول سؤال (كبير؟) أيوه، والباقي لأ. `?` : التلاتة لأ. في البرنامج، الأسئلة بتتسأل بالترتيب في سلسلة `if / else if / else` :

```
if (isupper(inputChar)) cout << "Uppercase Letter" << endl;
else if (islower(inputChar)) cout << "Lowercase Letter" << endl;
else if (isdigit(inputChar)) cout << "Number" << endl;
else cout << "Invalid Input" << endl;
```

أول ما شرط يطلع صح، الرسالة بتاعته بتتطبع، والسلسلة كلها بتخلص، والشروط اللي بعده ما بتتسألش أصلاً. ولو كل الشروط طلعت غلط، الـ `else` الأخيرة هي اللي تشتغل. فمستحيل يطلع أكتر من رسالة، ومستحيل ما يطلعش ولا رسالة.

نغير الإدخال بس

نفس البرنامج، وكل مرة حرف مختلف، علشان كل فرع يشتغل مرة:

الإدخال

```
A
```

الخرج

```
Enter a single character (uppercase or lowercase letter or a digit 0:9): Uppercase Letter
```

الإدخال

```
z
```

الخرج

```
Enter a single character (uppercase or lowercase letter or a digit 0:9): Lowercase Letter
```

الإدخال

```
9
```

الخرج

```
Enter a single character (uppercase or lowercase letter or a digit 0:9): Number
```

الإدخال

```
?
```

الخرج

```
Enter a single character (uppercase or lowercase letter or a digit 0:9): Invalid Input
```

الحرف `'0'` مش هو صفر السؤال 1

في السؤال 1 كنا بنقرا في `int` ، والصفر كان رقم معناه «وقّف». هنا بنقرا في `char` ، فالـ `0` اللي بيتكتب هو الحرف `'0'` ، وكوده 48:

```cpp
char c = '0';
int n = 0;
cout << int(c) << " " << n << endl;
cout << (isdigit(c) != 0) << endl;
```

الخرج

```
48 0
1
```

الحرف `'0'` كوده `48`، والرقم `0` قيمته صفر، حاجتين مختلفين. و `isdigit` قالت عن الحرف ده «أيوه رقم». فالبرنامج بيصنفه `Number` عادي، ومفيش أي معنى للوقوف هنا:

الإدخال

```
0
```

الخرج

```
Enter a single character (uppercase or lowercase letter or a digit 0:9): Number
```

البرنامج كامل

```cpp
#include <iostream>
#include <cctype> // Required for character handling functions
using namespace std;

int main() {
    char inputChar;

    // 1. read a single character
    cout << "Enter a single character (uppercase or lowercase letter or a digit 0:9): ";
    if (!(cin >> inputChar)) return 0;

    // 2. classify the entered character using cctype functions
    if (isupper(inputChar)) cout << "Uppercase Letter" << endl;
    else if (islower(inputChar)) cout << "Lowercase Letter" << endl;
    else if (isdigit(inputChar)) cout << "Number" << endl;
    else cout << "Invalid Input" << endl;

    return 0;
}
```

حل تاني: مقارنة المدى

بدل دوال `<cctype>`، نسأل عن الكود نفسه: هل هو بين حدين؟

```
if (inputChar >= 'A' && inputChar <= 'Z') cout << "Uppercase Letter" << endl;
else if (inputChar >= 'a' && inputChar <= 'z') cout << "Lowercase Letter" << endl;
else if (inputChar >= '0' && inputChar <= '9') cout << "Number" << endl;
else cout << "Invalid Input" << endl;
```

`inputChar >= 'A' && inputChar <= 'Z'` معناها «الحرف من A لـ Z». لازم `&&` لأن الحرف لازم يبقى بعد الحد الأول وقبل الحد التاني في نفس الوقت. البديل طلع نفس النتيجة في كل الإدخالات الأساسية. وجربنا الاتنين على جيران الحدود اللي في الجدول فوق، والاتنين قالوا `Invalid Input` لكل واحد. مثلاً `@` اللي قبل `A` على طول، و `:` اللي بعد `9` على طول:

الإدخال

```
@
```

الخرج

```
Enter a single character (uppercase or lowercase letter or a digit 0:9): Invalid Input
```

الإدخال

```
:
```

الخرج

```
Enter a single character (uppercase or lowercase letter or a digit 0:9): Invalid Input
```

الطريقة دي بتوضح فكرة المدى، لكنها معتمدة على ترتيب ASCII. دوال `<cctype>` مش محتاجة تعرفوا الأرقام دي خالص.

البرنامج البديل كامل

```
#include <iostream>
using namespace std;

int main() {
    char inputChar;

    // 1. read a single character
    cout << "Enter a single character (uppercase or lowercase letter or a digit 0:9): ";
    if (!(cin >> inputChar)) return 0;

    // 2. classify the entered character using ASCII ranges
    if (inputChar >= 'A' && inputChar <= 'Z') cout << "Uppercase Letter" << endl;
    else if (inputChar >= 'a' && inputChar <= 'z') cout << "Lowercase Letter" << endl;
    else if (inputChar >= '0' && inputChar <= '9') cout << "Number" << endl;
    else cout << "Invalid Input" << endl;

    return 0;
}
```

ممكن تقع في ده

- **تكتب `if` منفصلة لكل نوع.** كل `if` بتتسأل لوحدها، والـ `else` بتترتبط بآخر واحدة بس:

غلط مقصود: `if` منفصلة بدل `else if`

```
char c;
cin >> c;
if (isupper(c)) cout << "Uppercase Letter" << endl;
if (islower(c)) cout << "Lowercase Letter" << endl;
if (isdigit(c)) cout << "Number" << endl;
else cout << "Invalid Input" << endl;
```

الإدخال

```
A
```

الخرج

```
Uppercase Letter
Invalid Input
```

`A` طلّعت رسالتين متناقضتين، لأن `else` كانت تبع `isdigit` بس. لازم `else if`.

- **تكتب `||` في المدى.** `c >= 'A' || c <= 'Z'` صح لأي حرف في الدنيا، لأن أي كود يا من 65 وطالع، يا من 90 ونازل.
- **تقارن بـ `"A"`.** العلامتين نص، مش حرف. المقارنة لازم تبقى بـ `'A'`.
- **تكتب رقم من خانتين.** البرنامج بيقرا حرف واحد بس، فلو اتكتب `12` هياخد `1` ويقول `Number`، والـ `2` بتفضل في الإدخال:

الإدخال

```
12
```

الخرج

```
Enter a single character (uppercase or lowercase letter or a digit 0:9): Number
```

ده مش معناه إن 12 «رقم واحد». السؤال بيفترض حرف واحد بس.

- **تفتكر إن صفر هنا معناه وقّف.** ده كان في السؤال 1 مع `int`. هنا `'0'` حرف رقم عادي.

حل المصدر بنفس الشروط ونفس الترتيب ونفس الرسايل. البرنامج هنا زود بس سطر بيقفل لو القراية فشلت.

14 الحروف من A لـ Z، خمسة في كل سطر

(14) Write a C++ program that prints all the uppercase characters of the ASCII character table. Display 5 characters per line. Note that ASCII for $A = 65$ and $Z = 90$.

المطلوب

البرنامج يطبع كل الحروف الكبيرة في جدول ASCII، من `A` لحد `Z`، خمس حروف في كل سطر. السؤال نفسه بيدينا الأكواد: A = 65 و Z = 90، والاتنين داخلين. مفيش إدخال من المستخدم.

الخلفية

الحروف الكبيرة ورا بعض في الجدول من 65 لـ 90، يعني أي كود في المدى ده حرف كبير، ومفيش أي حاجة تانية في النص. وعدد الحروف 26، والـ 1 + 5 × 5 = 26. يعني هيبقى فيه خمس سطور كاملة، وسطر سادس فيه حرف واحد بس.

من C++ هنحتاج

حلقة `for` على أكواد `int`، وتحويل كل كود لـ `char` بـ `static_cast<char>` زي السؤال 10، وعداد للحروف اللي اتطبعت، ونفس قاعدة المسافات والسطور اللي في السؤال 8.

نبني الحل خطوة خطوة

> كل كود صغير في الجزء ده بيتجرب لوحده: حطوه مكان السطر `// one short example here` جوه `main` في الإطار اللي في أول الصفحة، في برنامج جديد لكل مثال. ولو تحت الكود فيه «الإدخال»، اكتبوه لما البرنامج يستناكم.

كود ولا حرف

```
int i = 65;
cout << i << " " << static_cast<char>(i) << endl;
```

الخرج

```
65 A
```

نفس المتغير `i` اتطبع مرة رقم ومرة حرف. الفرق كله في التحويل لـ `char`.

نلف على كل الأكواد

```
for (int i = 65; i <= 90; ++i)
    cout << static_cast<char>(i);
cout << endl;
```

الخرج

```
ABCDEFGHIJKLMNOPQRSTUVWXYZ
```

الحلقة لفت من 65 لحد 90 وطبعت كل حرف. الـ 26 حرف لازقين في سطر واحد لأننا ما طبعناش لا مسافات ولا سطور. قبل ما نزود ده، نتأكد من العدد:

```
int count = 0;
for (int i = 65; i <= 90; ++i)
    count++;
cout << count << endl;
```

الخرج

```
26
```

26 لفة، حرف لكل لفة. وده معناه إن `90 =>` صح: لو كتبناها `90 >` كانت هتقف قبل `Z` .

نضيف المسافات والسطور

نفس القاعدة اللي في السؤال 8، مع فرق واحد: هنا كل مرشح بيتطبع، لأن كل كود من 65 لـ 90 حرف كبير. فالعداد `char_per_line` بيزيد في كل لفة:

- قبل الحرف: لو العداد مش مضاعف للخمسة، نطبع مسافة.
- نطبع الحرف ونزود العداد.
- لو العداد بقى مضاعف للخمسة، ننزل سطر.
- بعد الحلقة: لو آخر سطر ناقص، ننزل سطر.

البرنامج كامل

```cpp
#include<iostream>
// #include <cmath>
using namespace std;

int main() {
    int char_per_line = 0;
    // 1. visit each candidate in the requested range
    for (int i = 65; i <= 90; ++i)
    {
        // 2. separate letters and track the output row
        if (char_per_line % 5 != 0) cout << " ";
        cout << static_cast<char>(i); // Print the character
        char_per_line++;
        // Check if 5 characters have been printed in the current line
        if (char_per_line % 5 == 0) cout << endl; // Move to the next line
    }

    if (char_per_line % 5 != 0) cout << endl;
    return 0;
}
```

الخرج

```
A B C D E
F G H I J
K L M N O
P Q R S T
U V W X Y
Z
```

خمس سطور كاملة، وسطر أخير فيه `Z` لوحدها. السطر الأخير ده انتهى بسطر جديد بفضل الـ `if` اللي بعد الحلقة، لأن العداد وقتها 26، ومش مضاعف للخمسة.

حل تاني: الحلقة على `char`

بما إن الحروف ورا بعض، ممكن العداد نفسه يبقى `char` ويبدأ من `'A'`:

```cpp
for (char letter = 'A'; letter <= 'Z'; ++letter)
{
    // 2. separate letters and track the output row
    if (char_per_line % 5 != 0) cout << " ";
    cout << letter; // Print the character
    char_per_line++;
    // Check if 5 characters have been printed in the current line
    if (char_per_line % 5 == 0) cout << endl; // Move to the next line
}
```

`++letter` بتنقل للكود اللي بعده، يعني الحرف اللي بعده. ومش محتاجين تحويل خالص، لأن المتغير `char` أصلاً، فـ `cout` بيطبعه حرف. البديل طالع نفس الست سطور بالظبط. والطريقة دي بتقرا أوضح («من A لـ Z»)، لكنها معتمدة إن الحروف ورا بعض في ASCII، وده صح هنا.

البرنامج البديل كامل

```
#include<iostream>
// #include <cmath>
using namespace std;

int main() {
    int char_per_line = 0;
    // 1. visit each candidate in the requested range
    for (char letter = 'A'; letter <= 'Z'; ++letter)
    {
        // 2. separate letters and track the output row
        if (char_per_line % 5 != 0) cout << " ";
        cout << letter; // Print the character
        char_per_line++;
        // Check if 5 characters have been printed in the current line
        if (char_per_line % 5 == 0) cout << endl; // Move to the next line
    }

    if (char_per_line % 5 != 0) cout << endl;
    return 0;
}
```

ممكن تقع في ده

- **تكتب `90 <`.** آخر حرف هيبقى `Y`:

غلط مقصود: `90 <` بدل `90 <=`

```
char last = ' ';
for (int i = 65; i < 90; ++i)
    last = static_cast<char>(i);
cout << last << endl;
```

الخرج

```
Y
```

السؤال قال A لـ Z، والاتنين داخلين.

- **تنسى التحويل.** `cout << i` هتطبع 65 و 66 و 67... مش الحروف.
- **تطبع مسافة بعد كل حرف وما تنزلش السطر الأخير.** ده اللي في حل المصدر (تحت).
- **تعدّ من الصفر في الشرط.** العداد بيبدأ صفر، وبيوصل 5 بعد خامس حرف. لو اختبرت `% 5 == 0` *قبل* ما تزوده، هتنزل سطر قبل أول حرف.

ملاحظة على حل المصدر

حل المصدر بيطبع الحروف الصح، لكنه بيطبع مسافة بعد كل حرف، حتى آخر حرف في كل سطر، وما بينزلش سطر بعد `Z`. ده نفس طريقته، مع `|` في آخر كل سطر علشان المسافة تبان، و `|` بعد الحلقة علشان نشوف آخر حاجة اتطبعت:

للمقارنة: طريقة حل المصدر، مع | علشان المسافات تبان

```
int char_per_line = 0;
for (int i = 65; i <= 90; ++i)
{
    cout << static_cast<char>(i) << " ";
    char_per_line++;
    if (char_per_line % 5 == 0) cout << "|" << endl;
}
cout << "|" << endl;
```

الخرج

```
A B C D E |
F G H I J |
K L M N O |
P Q R S T |
U V W X Y |
Z |
```

البرنامج هنا بيحط المسافة بين الحروف بس، وبينزل سطر بعد السطر الأخير الناقص.

---

15 أول خمس مضاعفات لعدد

(15) Write a C++ program that reads an integer and displays its first five multiples. That is in the input number is N the program displays $\times 1$, $N \times 2$, $N \times 3$, $N \times 4$, $N \times 5$

المطلوب

البرنامج يقرا عدد صحيح، ونسميه N، ويطبع أول خمس مضاعفات ليه: N × 1، و N × 2، و N × 3، و N × 4، و N × 5، في سطر واحد وبينهم فاصلة ومسافة. العدد ممكن يبقى موجب أو صفر أو سالب، لأن السؤال قال «an integer» بس.

الخلفية

مضاعف العدد N هو N مضروب في عدد صحيح. يعني N ثابت، واللي بيتغير هو اللي بنضرب فيه: 1، 2، 3، 4، 5. خلونا نفرق بين المضاعفات وحاجتين شبهها في الشكل، على N = 7:

مقارنة: السطر الأول بس هو المضاعفات المطلوبة

```
int N = 7;
for (int i = 1; i <= 5; ++i) cout << N * i << " ";
cout << endl;
int power = 1;
for (int i = 1; i <= 5; ++i)
{
    power = power * N;
    cout << power << " ";
}
cout << endl;
for (int i = 0; i < 5; ++i) cout << N + i << " ";
cout << endl;
```

الخرج

```
7 14 21 28 35
7 49 343 2401 16807
7 8 9 10 11
```

- السطر الأول: المضاعفات. كل مرة `N * i`، و `i` بيزيد. الفرق بين أي رقمين ورا بعض ثابت، وهو N نفسه.
- السطر التاني: القوى (powers). كل مرة بنضرب النتيجة اللي فاتت في N تاني. الأرقام بتكبر بسرعة جداً. ده مش المطلوب.
- السطر التالت: أعداد ورا بعض بتبدأ من N. بتزيد واحد بس. ده كمان مش المطلوب.

من C++ هنحتاج

`cin` لقراية N، وحلقة `for` من 1 لـ 5، وضرب `N * i`، و `if` صغيرة علشان الفاصلة ما تتطبعش بعد آخر رقم.

نبني الحل خطوة خطوة

> كل كود صغير في الجزء ده بيتجرب لوحده: حطوه مكان السطر `// one short example here` جوه `main` في الإطار اللي في أول الصفحة، في برنامج جديد لكل مثال. ولو تحت الكود فيه «الإدخال»، اكتبوه لما البرنامج يستناكم.

الحساب الأول، وبعدين الفواصل

السطر الأول في المثال فوق هو الحساب كله: N ثابت على 7، والمضروب فيه بيتغير من 1 لـ 5. فاضل شكل الطباعة. أول محاولة: نطبع `", "` بعد كل رقم، و `|` في الآخر علشان نشوف نهاية السطر:

غلط مقصود: فاصلة بعد آخر رقم

```
int N = 7;
for (int i = 1; i <= 5; ++i)
    cout << N * i << ", ";
cout << "|" << endl;
```

الخرج

```
7, 14, 21, 28, 35, |
```

بعد `35` فيه فاصلة ومسافة زيادة. الفاصلة مكانها بين الأرقام، يعني بعد كل رقم ما عدا الأخير. نغير حاجة واحدة: الفاصلة تتطبع بس لو `i < 5` :

```
int N = 7;
for (int i = 1; i <= 5; ++i)
{
    cout << N * i;
    if (i < 5) cout << ", ";
}
cout << "|" << endl;
```

الخرج

```
7, 14, 21, 28, 35|
```

دلوقتي `35` لازقة في `|` . الشرط `i < 5` صح في أول أربع لفات وغلط في الخامسة بس، وهي آخر رقم.

البرنامج كامل

```cpp
#include<iostream>
// #include <cmath>
using namespace std;

int main() {
    int N;

    // 1. read an integer
    cout << "Enter an integer: ";
    if (!(cin >> N)) return 0;

    cout << "The first five multiples of " << N << " are: ";
    // 2. multiply the input by each requested multiplier
    for (int i = 1; i <= 5; ++i)
    {
        cout << (N * i);
        if (i < 5) cout << ", ";
    }
    cout << endl;
    return 0;
}
```

N موجب، وصفر، وسالب:

الإدخال

```
7
```

الخرج

```
Enter an integer: The first five multiples of 7 are: 7, 14, 21, 28, 35
```

الإدخال

```
0
```

الخرج

```
Enter an integer: The first five multiples of 0 are: 0, 0, 0, 0, 0
```

الإدخال

```
-3
```

الخرج

```
Enter an integer: The first five multiples of -3 are: -3, -6, -9, -12, -15
```

الصفر مضروب في أي حاجة بصفر، فالخمس مضاعفات أصفار، وده صح. والسالب مضاعفاته سالبة وبتبعد عن الصفر. البرنامج ما احتاجش أي حالة خاصة لده.

وفيه حد لازم تعرفوه: `N * i` بيتحسب كـ `int` ، والـ `int` ليه أكبر قيمة. لو N كبير جداً، الضرب ممكن يعدّي الحد ده والنتيجة تبقى غلط. البرنامج مفترض إن N في حدود عادية بحيث `N * 5` يفضل جوه `int` .

حل تاني: `while`

```
int i = 1;
while (i <= 5)
{
    cout << (N * i);
    if (i < 5) cout << ", ";
    i++;
}
```

نفس الأجزاء: البداية قبل الحلقة، والشرط `i <= 5` ، والزيادة في آخر الجسم بعد الطباعة. وشرط الفاصلة هو هو. البديل طلع نفس الخرج في التلات إدخالات.

البرنامج البديل كامل

```
#include<iostream>
// #include <cmath>
using namespace std;

int main() {
    int N;

    // 1. read an integer
    cout << "Enter an integer: ";
    if (!(cin >> N)) return 0;

    cout << "The first five multiples of " << N << " are: ";
    // 2. multiply the input by each requested multiplier
    int i = 1;
    while (i <= 5)
    {
        cout << (N * i);
        if (i < 5) cout << ", ";
        i++;
    }
    cout << endl;
    return 0;
}
```

ممكن تقع في ده

- **تبدأ `i` من صفر.** أول رقم هيبقى 0 = 0 × N، وده مش من «أول خمس مضاعفات» المطلوبة، اللي بتبدأ بـ 1 × N.

- **تضرب النتيجة في N كل مرة.** ده بيطلّع القوى (السطر التاني في المثال)، مش المضاعفات.
- **تزود واحد بدل ما تضرب.** ده بيطلّع أعداد ورا بعض (السطر التالت).
- **تطبع `", "` بعد كل رقم.** هتفضل فاصلة زيادة في الآخر.
- **تكتب الشرط `i <= 5` للفاصلة.** ده صح في كل اللغات، فالفاصلة هتتطبع بعد الأخير برضه. لازم `i < 5` .

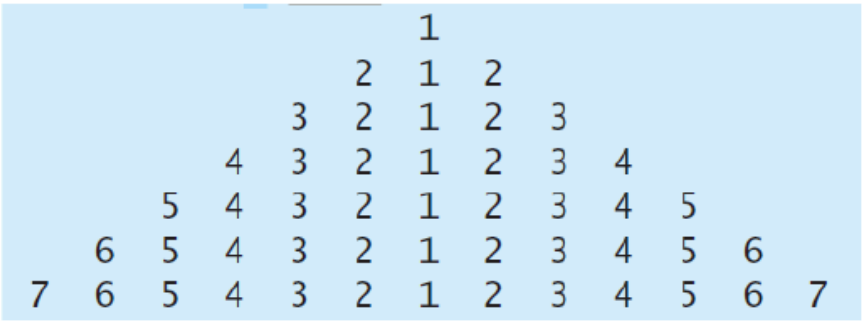
ملاحظة على الشيت

نص السؤال في الشيت بيبدأ قايمة المضاعفات بـ «1 ×» من غير الـ N اللي قبلها، والباقي مكتوب كامل (N × 2، N × 3، ...). المقصود واضح من باقي القايمة ومن جملة «first five multiples»: أول حاجة هي N × 1. حل المصدر بيحسب كده، والبرنامج هنا زيه.

---

16 هرم الأرقام

(16) Write a C++ program that prompts the user to enter an integer from 1 to 15 and displays a pyramid like that one shown in the below figure.



المطلوب

البرنامج يطلب عدد صحيح N من 1 لـ 15، ويطبع هرم ارتفاعه N سطر. السطر رقم i فيه الأرقام نازلة من i لحد 1، وبعدين طالعة من 2 لحد i، والهرم كله متوسّط بحيث الـ 1 اللي في كل سطر يبقى تحت بعض. الصورة في الشيت لـ N = 7. ولو الرقم بره المدى، البرنامج يقول كده وما يرسمش حاجة.

الخلفية

بصوا على سطر واحد من غير ما تفكروا في الهرم كله. السطر رقم 4 من هرم N = 7 مقسوم لتلات حتت، وكل حتة متلونة بلون (السطر نفسه متاخد من الخرج المسجل تحت):

```
    4 3 2 1 2 3 4
```



- **المسافات في الأول:** السطر 4 فيه أرقام أقل من آخر سطر، فبيتزق ناحية اليمين علشان يتوسّط. كل سطر فوق آخر سطر بواحد بيتزق خانة واحدة. والسطر i بيتزق N − i خانة، يعني هنا 3 = 4 − 7 خانات.
- **النازل:** من 4 لـ 1، أربع أرقام. الـ 1 في الآخر هو نص السطر.
- **الطالع:** من 2 لـ 4، تلات أرقام بس. بيبدأ من 2 مش من 1، لأن الـ 1 اتطبع خلاص في النازل.

يعني السطر i فيه 1 − 2i = (i − 1) + i رقم، والنص بيتطبع مرة واحدة بس.

من C++ هنحتاج

حلقات جوه حلقات (nested loops): حلقة بره بتختار رقم السطر، وتلات حلقات جوه بتكمّل السطر ده. ومعاهم `setw(3)` من `<iomanip>` علشان كل رقم ياخد نفس العرض، و`if` بـ `||` للمدى. و`cerr` لرسالة الخطأ.

نبني الحل خطوة خطوة

> كل كود صغير في الجزء ده بيتجرب لوحده: حطوه مكان السطر `// one short example here` جوه `main` في الإطار اللي في أول الصفحة، في برنامج جديد لكل مثال. ولو تحت الكود فيه «الإدخال»، اكتبوه لما البرنامج يستناكم.

كل رقم في خانة عرضها 3

علشان الأعمدة تبقى تحت بعض، لازم كل رقم ياخد نفس عدد الأماكن، سواء كان رقم واحد أو رقمين:

```
cout << "|" << setw(3) << 7 << "|" << setw(3) << 10 << "|" << endl;
cout << "|" << "   " << "|" << endl;
```

الخرج

```
|  7| 10|
|   |
```

`7` اتكملت بمسافتين، و`10` بمسافة واحدة، والاتنين بقوا 3 أماكن. والسطر التاني بيبين إن `"   "` (تلات مسافات) بتاخد نفس المكان بالظبط. علشان كده كل «خانة فاضية» في أول السطر بتتطبع تلات مسافات، مش مسافة واحدة: الخانة الفاضية لازم تبقى بعرض الخانة اللي فيها رقم.

الحتت التلاتة بعلامات

نكتب الهرم لـ N = 3، وبدل المسافات اللي في الأول نطبع نقط، ونحط أقواس حوالين النازل وحوالين الطالع، علشان نشوف كل حلقة طبعت إيه:

```cpp
int N = 3;
for (int i = 1; i <= N; ++i)
{
    for (int j = 0; j < N - i; ++j) cout << "...";
    cout << "[";
    for (int k = i; k >= 1; --k) cout << setw(3) << k;
    cout << "][";
    for (int l = 2; l <= i; ++l) cout << setw(3) << l;
    cout << "]" << endl;
}
```

الخرج

```
......[  1][]
...[  2  1][  2]
[  3  2  1][  2  3]
```

- الحلقة بره `i` بتمشي 1 و 2 و 3، سطر لكل لفة.
- حلقة النقط `j` بتلف `N - i` مرة: مرتين في السطر الأول، ومرة في التاني، وولا مرة في التالت.
- حلقة النازل `k` بتبدأ من `i` وتنزل لـ 1.
- حلقة الطالع `l` بتبدأ من 2 وتطلع لـ `i` . في السطر الأول، 2 أكبر من 1 من الأول، فالحلقة ما لفتش ولا مرة، والقوسين فضيوا `[]` .

وأهم حاجة: كل حلقة جوه بتبدأ من الأول في كل سطر. `k` في السطر التالت بدأ من 3، مش كمّل من المكان اللي وقف فيه في السطر اللي قبله. ده بيحصل لوحده لأن `int k = i` مكتوبة جوه `for` ، فبتتنفذ من جديد كل مرة الحلقة بتبدأ.

ليه الطالع بيبدأ من 2

نغير حاجة واحدة: الطالع يبدأ من 1.

غلط مقصود: الطالع بيبدأ من 1

```cpp
int N = 3;
for (int i = 1; i <= N; ++i)
{
    for (int j = 0; j < N - i; ++j) cout << "   ";
    for (int k = i; k >= 1; --k) cout << setw(3) << k;
    for (int l = 1; l <= i; ++l) cout << setw(3) << l;
    cout << endl;
}
```

الخرج

```
      1  1
    2  1  1  2
  3  2  1  1  2  3
```

الـ 1 بقى متكرر في نص كل سطر، لأن النازل طبعه، والطالع طبعه تاني. البداية من 2 هي اللي بتخلي النص يتطبع مرة واحدة.

البرنامج كامل

```cpp
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    int N;

    cout << "Enter an integer from 1 to 15: ";
    if (!(cin >> N)) return 0;
    if (N < 1 || N > 15)
    {
        cerr << "Invalid input. Please enter an integer between 1 and 15." << endl;
        return 0; // The invalid range has been handled
    }

    // 2. build each pyramid row
    for (int i = 1; i <= N; ++i)
    {
        // Print leading spaces
        for (int j = 0; j < N - i; ++j) cout << "   ";

        // Print numbers in decreasing order
        for (int k = i; k >= 1; --k) cout << setw(3) << k;

        // Print numbers in increasing order (excluding 1 to avoid duplication)
        for (int l = 2; l <= i; ++l) cout << setw(3) << l;

        cout << endl;
    }
    return 0;
}
```

نفس مثال الشيت (N = 7). أول سطر بيبدأ بعد رسالة الطلب على طول، لأن الإدخال اتبعت جاهز:

الإدخال

```
7
```

الخرج

```
Enter an integer from 1 to 15:                   1
                  2  1  2
               3  2  1  2  3
            4  3  2  1  2  3  4
         5  4  3  2  1  2  3  4  5
      6  5  4  3  2  1  2  3  4  5  6
   7  6  5  4  3  2  1  2  3  4  5  6  7
```

أصغر قيمتين:

الإدخال

```
1
```

الخرج

```
Enter an integer from 1 to 15:   1
```

الإدخال

```
2
```

الخرج

```
Enter an integer from 1 to 15:      1
  2  1  2
```

وأكبر قيمة مسموحة، N = 15. هنا بتظهر قيمة `setw(3)` : من السطر العاشر الأرقام بقت من خانتين، ومع كده الأعمدة فضلت تحت بعض، لأن كل رقم لسه واخد 3 أماكن. وآخر سطر بيبدأ بمسافة واحدة بس قبل `15` ، اللي هي اللي `setw(3)` كملتها:

الإدخال

```
15
```

الخرج

```
Enter an integer from 1 to 15:                                        1
                                      2  1  2
                                   3  2  1  2  3
                                4  3  2  1  2  3  4
                             5  4  3  2  1  2  3  4  5
                          6  5  4  3  2  1  2  3  4  5  6
                       7  6  5  4  3  2  1  2  3  4  5  6  7
                    8  7  6  5  4  3  2  1  2  3  4  5  6  7  8
                 9  8  7  6  5  4  3  2  1  2  3  4  5  6  7  8  9
              10  9  8  7  6  5  4  3  2  1  2  3  4  5  6  7  8  9 10
           11 10  9  8  7  6  5  4  3  2  1  2  3  4  5  6  7  8  9 10 11
        12 11 10  9  8  7  6  5  4  3  2  1  2  3  4  5  6  7  8  9 10 11 12
     13 12 11 10  9  8  7  6  5  4  3  2  1  2  3  4  5  6  7  8  9 10 11 12 13
  14 13 12 11 10  9  8  7  6  5  4  3  2  1  2  3  4  5  6  7  8  9 10 11 12 13 14
15 14 13 12 11 10  9  8  7  6  5  4  3  2  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
```

وبره المدى. الرسالة بتتطبع على `cerr` ، وده مكان رسايل الأخطاء، فهي متسجلة لوحدها تحت:

الإدخال

```
0
```

الخرج

```
Enter an integer from 1 to 15:
```

رسالة الخطأ (على `cerr`)

```
Invalid input. Please enter an integer between 1 and 15.
```

الإدخال

```
16
```

الخرج

```
Enter an integer from 1 to 15:
```

رسالة الخطأ (على `cerr`)

```
Invalid input. Please enter an integer between 1 and 15.
```

حل تاني: حلقات `while`

كل `for` في البرنامج ممكن تتكتب `while`. البديل بيعمل كده، وده جسم الحلقة بره:

```
int i = 1;
while (i <= N)
{
    int j = 0;
    while (j < N - i)
    {
        cout << "   ";
        j++;
    }

    int k = i;
    while (k >= 1)
    {
        cout << setw(3) << k;
        k--;
    }

    int l = 2;
    while (l <= i)
    {
        cout << setw(3) << l;
        l++;
    }

    cout << endl;
    i++;
}
```

البديل طلع نفس الخرج بالظبط في الست إدخالات. لكن فيه حاجة لازم تاخدوا بالكم منها في `while` : البداية (`int j = 0;` و `int k = i;` و `int l = 2;`) مكتوبة جوه الحلقة بره، فبتتنفذ من جديد كل سطر. لو نقلنا واحدة منهم بره، العداد مش هيرجع لأوله. ده نفس البرنامج لـ N = 4، بس `int l = 2;` اتنقلت قبل الحلقة بره:

غلط مقصود: `int l = 2;` اتنقلت بره الحلقة

```
int N = 4;
int i = 1;
int l = 2;
while (i <= N)
{
    int j = 0;
    while (j < N - i)
    {
        cout << "   ";
        j++;
    }
    int k = i;
    while (k >= 1)
    {
        cout << setw(3) << k;
        k--;
    }
    while (l <= i)
    {
        cout << setw(3) << l;
        l++;
    }
    cout << endl;
    i++;
}
```

الخرج

```
          1
       2  1  2
    3  2  1  3
 4  3  2  1  4
```

أول سطرين طلعوا صح بالصدفة. في السطر التالت `l` كان لسه 3 من اللي فات، فالطالع طبع `3` بس من غير `2` ، وفي السطر الرابع طبع `4` بس. في `for` الغلطة دي أصعب تحصل، لأن البداية مكتوبة في نفس سطر الحلقة.

البرنامج البديل كامل

```cpp
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    int N;

    cout << "Enter an integer from 1 to 15: ";
    if (!(cin >> N)) return 0;
    if (N < 1 || N > 15)
    {
        cerr << "Invalid input. Please enter an integer between 1 and 15." << endl;
        return 0; // The invalid range has been handled
    }

    // Build each row from spaces, its descending half, and its ascending half.
    int i = 1;
    while (i <= N)
    {
        int j = 0;
        while (j < N - i)
        {
            cout << "   ";
            j++;
        }

        int k = i;
        while (k >= 1)
        {
            cout << setw(3) << k;
            k--;
        }

        int l = 2;
        while (l <= i)
        {
            cout << setw(3) << l;
            l++;
        }

        cout << endl;
        i++;
    }
    return 0;
}
```

ممكن تقع في ده

- **تطبع مسافة واحدة للخانة الفاضية.** الخانة اللي فيها رقم عرضها 3، فالخانة الفاضية لازم تبقى 3 كمان:

غلط مقصود: مسافة واحدة للخانة الفاضية

```
int N = 4;
for (int i = 1; i <= N; ++i)
{
    for (int j = 0; j < N - i; ++j) cout << " ";
    for (int k = i; k >= 1; --k) cout << setw(3) << k;
    for (int l = 2; l <= i; ++l) cout << setw(3) << l;
    cout << endl;
}
```

الخرج

```
    1
   2  1  2
  3  2  1  2  3
 4  3  2  1  2  3  4
```

الهرم اتزق ناحية الشمال واتلخبط، لأن كل خانة فاضية بقت تلت عرضها.

- **تبدأ الطالع من 1.** النص هيتكرر، زي ما شفنا.
- **تعرّف عداد حلقة جوه بره الحلقة اللي بتحتاجه.** العداد مش هيرجع لأوله في السطر الجديد.
- **تكتب `setw(2)`.** الأرقام اللي من خانتين هتلزق في بعض:

السطر الأول غلط مقصود: `setw(2)`؛ التاني بـ `setw(3)` للمقارنة

```
cout << setw(2) << 11 << setw(2) << 10 << setw(2) << 9 << endl;
cout << setw(3) << 11 << setw(3) << 10 << setw(3) << 9 << endl;
```

الخرج

```
1110 9
 11 10  9
```

السطر الأول مش مفهوم فين رقم بيخلص وفين التاني بيبدأ. `setw(3)` بتضمن مسافة واحدة على الأقل قبل أي رقم لحد 15.

- **تنسى `endl` في آخر كل سطر.** الهرم كله هيبقى سطر واحد طويل.
- **تكتب المدى بـ `&&`.** `N < 1 && N > 15` مستحيل، فمش هترفض حاجة.

حل المصدر بنفس الحلقات التلاتة ونفس `setw(3)`. الفرق الوحيد إنه بيقفل بـ `return 1` لما الرقم يبقى بره المدى، والبرنامج هنا بيقفل بـ `return 0` بعد ما يطبع نفس الرسالة. وزود سطر بيقفل لو القراية فشلت.